

1.	Модули math и random.....	3
1.1.	Математический модуль math.....	3
1.2.	Модуль random	7
2.	Построение графиков с matplotlib	10
2.1.	Установка и подключение.....	10
2.2.	Построение графиков.....	10
2.3.	figure и axes	17
2.4.	Полярная система координат	21
2.5.	Графики с numpy	22
2.6.	Геометрические объекты.....	24
2.7.	Трехмерные графики	25
3.	Задачи на matplotlib.....	26
3.1.	Пример решения.....	26
3.2.	Условия	30
4.	Библиотека NumPy.....	32
4.1.	Массивы	32
4.2.	Операции над массивами	34
4.3.	Пакет SciPy	36
5.	ООП	37
5.1.	Объектно-ориентированное программирование.....	37
5.2.	Создание класса.....	38
5.3.	Специальные (магические) методы.....	39
5.4.	Инкапсуляция	42
5.5.	Наследование.....	44
5.6.	Полиморфизм	44

5.7. Абстрактные классы	45
5.8. Статические методы и методы класса	47
6. Аннотации.....	48
7. Исключения	50
7.1. Обработка исключений.....	50
7.2. Вызов исключений.....	52
7.3. Пользовательские исключения	53
8. Декораторы	53
8.1. Пользовательские декораторы.....	53
8.2. Применение декораторов	55
8.3. Встроенные декораторы	57
8.4. Декораторы классов	59
8.5. Декораторы с аргументами	60
9. Комплексные числа.....	61
10. Длинная арифметика.....	61
11. Множественный выбор match.....	62
12. Задачи	64
12.1. Примеры задач.....	67
12.2. Задачи на функции	73
12.3. Задачи на ООП	75
12.4. Задачи на исключения	77
12.5. Задачи на декораторы	78
13. Проверочные вопросы	Ошибка! Закладка не определена.
13.1. Вопросы	Ошибка! Закладка не определена.
13.2. Ответы	Ошибка! Закладка не определена.

1. Модули **math** и **random**

1.1. Математический модуль **math**

Модуль предоставляет обширный функционал для работы с числами. Некоторые его методы:

- тригонометрические и обратные тригонометрические функции;
- факториал;
- перевод градусов в радианы и обратно;
- логарифм;
- квадратный корень;
- модуль;
- округление
- другие.

Пример использования данного модуля:

```
1 | import math
2 |
3 | print(math.factorial(6)) # 720
4 | print(math.log(81, 3)) # 4.0
```

К методам округления относятся:

- функция **ceil**, которая округляет число до ближайшего целого в большую сторону,
- функция **floor**, которая округляет число до ближайшего целого в меньшую сторону,
- функция **trunc**, которая усекает значение до целого.

Пример использования функций округления:

```
1 | import math
2 |
3 | print(math.ceil(6.25)) # 7
4 | print(math.ceil(6.75)) # 7
5 | print(math.floor(6.25)) # 6
6 | print(math.floor(6.75)) # 6
7 | print(math.trunc(6.25)) # 6
8 | print(math.trunc(6.75)) # 6
```

Из данного примера может показаться, что функции `floor` и `trunc` повторяют друг друга, однако на отрицательных величинах разница видна наглядно:

```
1 | import math
2 |
3 | print(math.floor(-6.25)) # -7
4 | print(math.floor(-6.75)) # -7
5 | print(math.trunc(-6.25)) # -6
6 | print(math.trunc(-6.75)) # -6
```

Для привычного округления (на самом деле для «банковского») используется метод `round` не относящийся к модулю `math`:

```
1 | print(round(6.25)) # 6
2 | print(round(6.75)) # 7
```

Функция **`fabs`** используется для вычисления абсолютного значения числа. Данная функция всегда возвращает число с плавающей точкой:

```
1 | import math
2 |
3 | print(math.fabs(-5)) # 5.0
4 | print(math.fabs(-5.5)) # 5.5
```

Функция **`factorial`** принимает положительное целое число и выводит его факториал.

Функция **`fmod`** возвращает остаток от деления. Для чисел типа `float` данная функция является предпочтительнее чем оператор `%`, который в свою очередь является предпочтительным для чисел типа `int`.

Функция **`frexp`** возвращает мантиссу и показатель степени в виде пары (m, e) любого числа x , решая следующее уравнение: $x = m * 2^e$. Мантисса m представляет собой число с плавающей запятой, а показатель e представляет собой целочисленное значение.

Значение m вычисляется таким способом, чтобы его модуль лежал в полуинтервале $[0.5; 1)$.

Данная функция используется тогда, когда представление чисел типа `float` не должно зависеть от архитектуры машины.

Функция **ldexp** возвращает значение равное $m \cdot 2^e$, то есть является обратной к функции `frexp`:

```
1 | import math
2 |
3 | print(math.frexp(15)) # (0.9375, 4)
4 | print(math.ldexp(0.9375, 4)) # 15.0
```

Функция **modf** возвращает кортеж из двух чисел (`f`, `w`) где `f` это дробная, а `w` — целая часть исходного числа. Результат всегда имеет тип `float`:

```
1 | import math
2 |
3 | print(math.modf(3.14))
4 | # (0.140000000000000012, 3.0)
```

Функция **copysign** возвращает число с абсолютным значением первого аргумента, но со знаком второго. Возвращаемый результат всегда типа `float`.

Функция **isclose** позволяет проверить на равенство вещественные числа с учетом погрешности. Параметры `rel_tol` и `abs_tol` задают точность:

```
1 | import math
2 |
3 | x = 7
4 | y = 7.0001
5 |
6 | print(x == y) # False
7 | print(math.isclose(x, y)) # False
8 | print(math.isclose(x, y, abs_tol=0.001)) # True
```

Функция **lcm** возвращает наименьшее общее кратное указанных целочисленных аргументов. Если какой-либо из аргументов равен нулю, то возвращается нуль.

Функция **gcd** возвращает наибольший общий делитель указанных целочисленных аргументов.

Функция **nextafter** возвращает следующее значение float после аргумента 1 по направлению к аргументу 2 (для версий Python 3.9 и выше):

```
1 | import math
2 |
3 | print(math.nextafter(6.2, 20)) # 6.2000000000000001
4 | print(math.nextafter(6.2, 0)) # 6.199999999999999
```

Функция **fsum** вычисляет точную сумму значений с плавающей точкой в итерируемом объекте (например, последовательности):

```
1 | import math
2 |
3 | print(math.fsum((6.2, 20, 4.1522))) # 30.3522
```

Функция **exp** принимает один вещественный параметр возвращает экспоненту в данной степени.

Функция **log** находит логарифм числа по заданному основанию. Если оно не указано, то основанием считается «е».

Функция **pow** возводит число в степень.

Функция **sqrt** вычисляет квадратный корень числа.

В Python есть следующие тригонометрические функции:

- **sin** принимает радиан и возвращает его синус,
- **cos** принимает радиан и возвращает его косинус,
- **tan** принимает радиан и возвращает его тангенс,
- **asin** принимает один параметр и возвращает арксинус (обратный синус),
- **acos** принимает один параметр и возвращает арккосинус (обратный косинус),
- **atan** принимает один параметр и возвращает арктангенс (обратный тангенс),
- **sinh** принимает один параметр и возвращает гиперболический синус,

- `cosh` принимает один параметр и возвращает гиперболический косинус,
- `tanh` принимает один параметр и возвращает гиперболический тангенс,
- `asinh` принимает один параметр и возвращает обратный гиперболический синус,
- `acosh` принимает один параметр и возвращает обратный гиперболический косинус,
- `atanh` принимает один параметр и возвращает обратный гиперболический тангенс.

Функции преобразования углов:

- `degrees` конвертирует радиан в градусы,
- `radians` конвертирует градусы в радианы.

В Python есть следующие математические константы: **`pi`** и **`e`**.

Константа **`inf`** возвращает положительную бесконечность, значение которое является типом `float` и может присутствовать в математических выражениях.

```

1 | import math
2 |
3 | print(math.inf/2) # inf

```

Результатом будет также бесконечность.

Константа **`nan`** возвращает значение «не число» (NaN) которое является типом `float` и может присутствовать в математических выражениях. Оно не считается равным никакому другому числовому значению, включая само себя.

1.2. Модуль **`random`**

Модуль `random` предоставляет функции для:

- генерации случайных чисел (`randint`, `random`, `uniform`);
- случайного выбора элементов последовательности (`choice`);
- перемешивания последовательности (`shuffle`)
- другие.

Пример использования:

```

1 | from random import randint, uniform

```

```

2 |
3 | i = randint(1, 50)    # Целое
4 | f = uniform (1, 50)  # Вещественное

```

Метод **randint** в Python возвращает случайное целочисленное значение между двумя нижними и верхними пределами (включая оба ограничения), предоставленными, как два целочисленных параметра. То есть в случае из примера, значение *i* будет принадлежать отрезку [1; 50].

Повторение метода `randint()` дает разные случайные целые числа для каждого вызова, то есть следующий пример будет выводить различные случайные числа, а не одно повторяющееся случайное число:

```

1 | for i in range(5):
2 |     print(randint(0, 99))

```

Если второй параметр не будет больше либо равен первому, или параметры будут иметь не целочисленный тип, то возникнет исключение `ValueError`.

Метод **random** возвращает случайное число с плавающей точкой в промежутке [0.0, 1.0].

Метод **randrange** в Python используется для генерации случайного числа в диапазоне, указанном в его параметрах: первый — начальный номер, второй — конечный номер, третий — ширина (шаг), которая используется для пропуска числа в диапазоне. Последний параметр является необязательным, и, если он не указан, принимает значение по умолчанию равное единице:

```

1 | from random import randrange
2 | print(randrange(0, 5, 2)) # 0, 2 или 4

```

Функция `randrange` не включает параметр конца (верхнее ограничение) при генерации случайного целого числа.

Метод **uniform** — возвращает случайное число с плавающей точкой в указанном диапазоне, причем нижняя граница включена в этот диапазон, а верхняя нет.

В модуле `random` есть функция **choice**, используемая для выбора случайного элемента из списка или другой последовательности:

```
1 | from random import choice
2 |
3 | s = 'Я пытаюсь найти след'
4 | print(choice(s)) # Случайный символ строки s
```

Метод **sample** используется, когда требуется выбрать несколько элементов из заданной последовательности. Метод возвращает список уникальных элементов, которые были выбраны из указанной в параметрах последовательности. Выбираемое количество элементов также задается параметром:

```
1 | from random import sample
2 |
3 | s = 'Чего больше здесь нет'
4 | print(sample(s, 2)) # Список из 2 символов
```

Метод **choices** также используется, когда требуется выбрать несколько случайных элементов из заданной последовательности, но он позволяет повторять несколько раз один и тот же элемент:

```
1 | from random import choices
2 |
3 | s = 'Этот огненный плен и зола'
4 | print(choices(s, k = 2)) # Список из 2 символов
```

Метод **seed** используется для инициализации генератора псевдослучайных чисел в Python. Модуль `random` использует значение из `seed`, или отправной точки как основу для генерации случайного числа. Если значения `seed` нет в наличии, тогда система будет отталкиваться от текущего времени.

Метод **shuffle** используется для перемешивания данных списка или другой последовательности. Метод `shuffle` смешивает элементы списка на месте, то есть изменяет сам список:

```
1 | from random import shuffle
2 |
```

```

3 | s = [1, 2, 3, 4, 5]
4 | shuffle(s)
5 | print(s)

```

Важно то, что данный метод применяется для изменяемых последовательностей, то есть попытка применить его к строке или кортежу вызовет ошибку.

Функция **triangular** используется в генерации случайных вещественных чисел с плавающей точкой. При генерации значения применяется треугольное распределение вероятности.

2. Построение графиков с matplotlib

2.1. Установка и подключение

Matplotlib — это набор дополнительных модулей на языке Python для визуализации данных. В нем можно построить двумерные и трехмерные графики.

Установка matplotlib для ОС Windows возможна через командную строку:

```

1 | python -m pip install -U pip
2 | pip install matplotlib

```

Для построения несложных графиков достаточно подключить модуль pyplot:

```

1 | import matplotlib.pyplot as p

```

Где p — это пользовательский псевдоним модуля.

2.2. Построение графиков

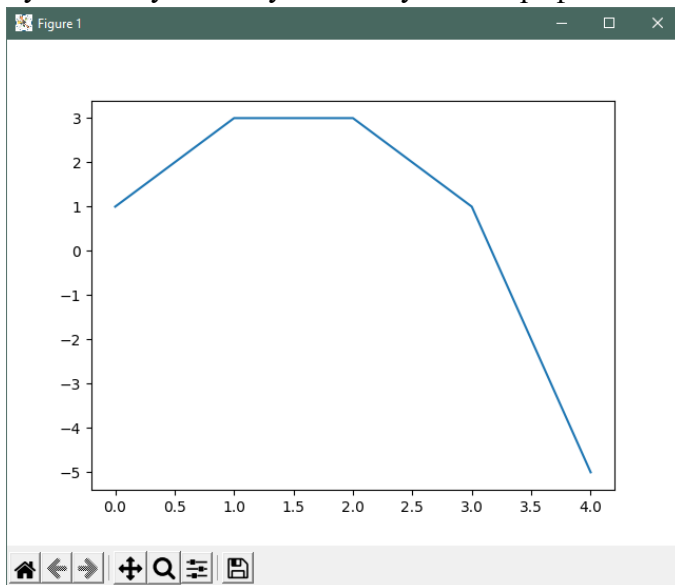
Метод **plot** — это метод построение графика, который принимает «иксы» и «игреки», show выводит построенное изображение:

```

1 | p.plot(list(range(5)), [1, 3, 3, 1, -5])
2 | p.show()

```

В результате будет получен следующий график.

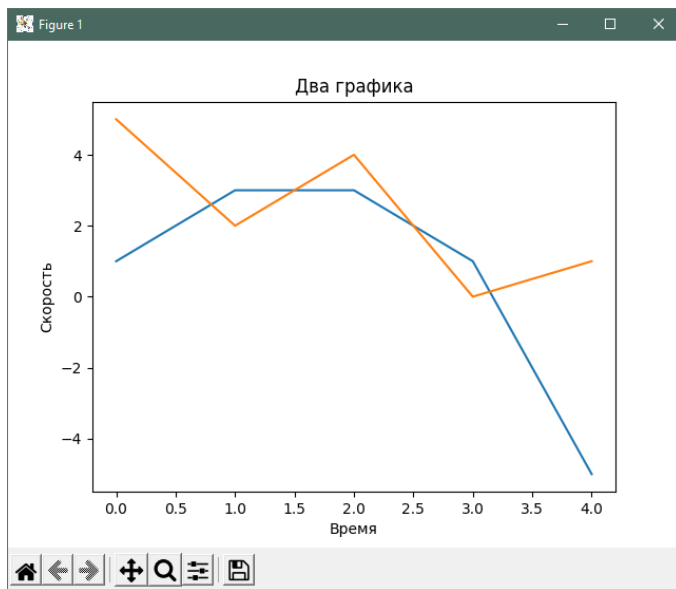


Добавляя в plot через запятую пары значений можно построить несколько графиков одновременно. Либо можно вызвать plot нужное число раз.

Можно указать **надпись** над графиком (заголовок) с помощью функции title. Чтобы подписать одну либо обе оси существуют функции xlabel и ylabel. Например:

```
1 | x = tuple(range(5))
2 | y1 = (1, 3, 3, 1, -5)
3 | y2 = (5, 2, 4, 0, 1)
4 | p.title("Два графика")
5 | p.xlabel("Время")
6 | p.ylabel("Скорость")
7 | p.plot(x, y1, x, y2)
8 | p.show()
```

В результате работы данного кода получаем следующие графики:



Цвет линий задается параметром `color`, значениями которого могут быть как коды цветов, так и их названия:

```
1 | x = tuple(range(5))
2 | y1 = (1, 3, 3, 1, -5)
3 | y2 = (5, 2, 4, 0, 1)
4 | p.plot(x, y1, color = (1.0, 0.0, 1.0)) # доли RGB
5 | p.plot(x, y2, color = '#df8856') # html-код цвета
```

Список распознаваемых по названию цветов содержит более 150 пунктов. При этом базовые цвета могут быть указаны одной буквой:

- b — синий,
- g — зеленый,
- r — красный,
- c — циан,
- m — манжета,
- y — желтый,

- k — черный,
- w — белый.

Параметр `linewidth` определяет **толщину линий**:

```
1 | p.plot(x, y1, linewidth = 6)
2 | p.plot(x, y2, linewidth = 3)
```

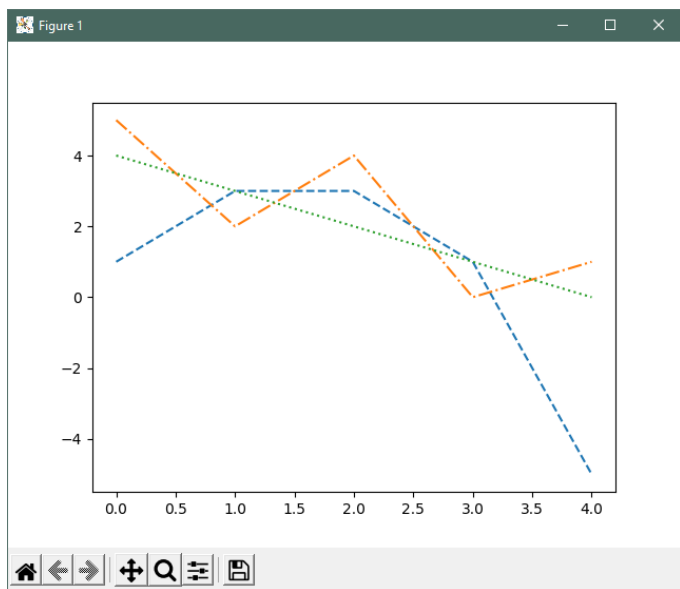
Тип линии задается параметром `linestyle`. Линия может быть:

- сплошной ('-', 'solid'),
- пунктирной ('--', 'dashed'),
- тире-точка ('-.', 'dashdot'),
- из точек (':', 'dotted'),
- не отображаться ('None', ' ', '').

В результате выполнения следующего кода

```
1 | x = tuple(range(5))
2 | y1 = (1, 3, 3, 1, -5)
3 | y2 = (5, 2, 4, 0, 1)
4 | y3 = list(range(5))[:-1]
5 | p.plot(x, y1, linestyle = '--')
6 | p.plot(x, y2, linestyle = '-.')
7 | p.plot(x, y3, linestyle = ':')
```

будут получены графики с пунктирной, сплошной и точечной линиями:

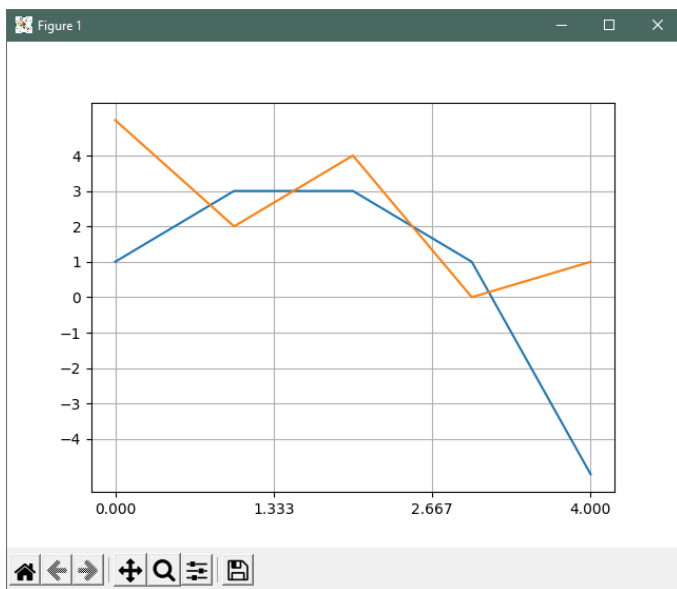


Чтобы просто включить или выключить **сетку** достаточно вызвать функцию `grid`, указав булевский аргумент — отображать сетку или нет. Цвет, толщина и тип линий сетки можно настраивать, аналогично тому, как это настраивается для самих графиков.

Чтобы настроить пользовательские значения вдоль осей (метки), а вместе с ними и ячейки сетки, используются функции `xticks` и `yticks`. Например:

```
1 | x = tuple(range(5))
2 | y1 = (1, 3, 3, 1, -5)
3 | y2 = (5, 2, 4, 0, 1)
4 | p.plot(x, y1, x, y2)
5 |
6 | p.grid(True) # Сетка
7 | p.xticks([0, 4/3, 8/3, 4])
8 | p.yticks(range(-4, 5, 1))
```

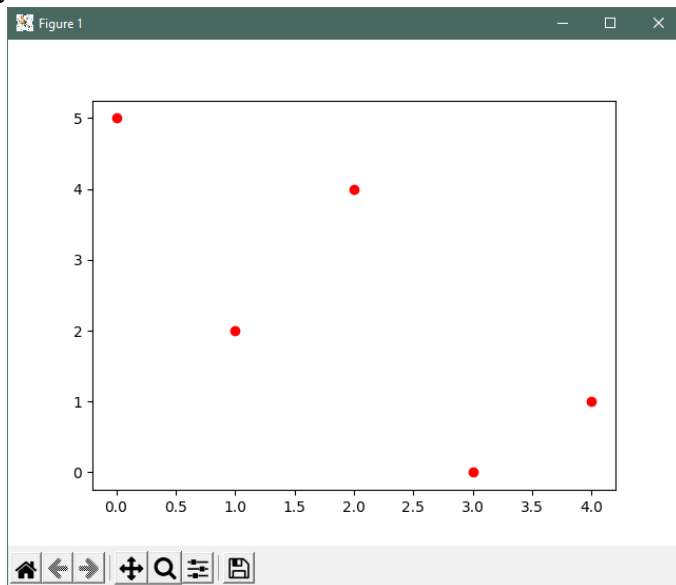
В результате сетка примет вид:



Метод **scatter** служит для построения точек, не соединяя их:

```
1 | p.scatter(x, y2, color = 'r')  
2 | p.show()
```

Результат:

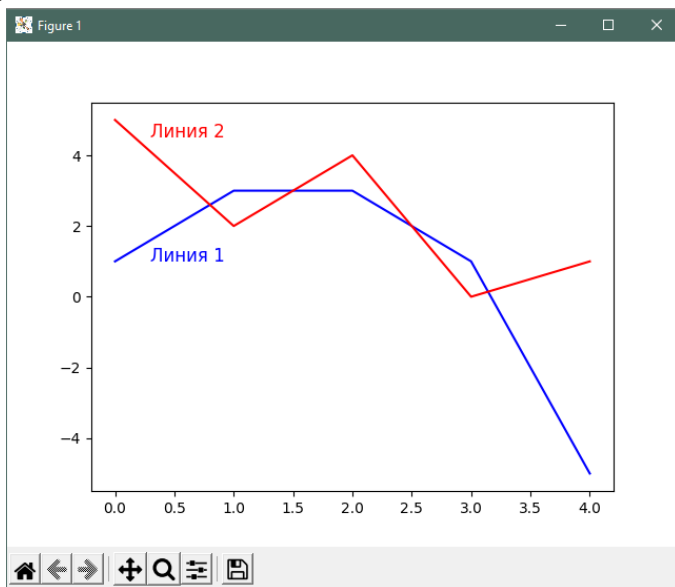


Добавление **текста** прямо на рисунок осуществляются функцией `text` с заданными координатами и отображаемой строкой.

Многие параметры настраиваются, например:

```
1 | x = tuple(range(5))
2 | y1 = (1, 3, 3, 1, -5)
3 | y2 = (5, 2, 4, 0, 1)
4 | p.plot(x, y1, color='b')
5 | p.plot(x, y2, color='r')
6 |
7 | p.text(0.3, 1, 'Линия 1',
8 |        fontsize=12, # Размер
9 |        color='b') # Цвет
10 | p.text(0.3, 4.5, 'Линия 2',
11 |        fontsize=12,
12 |        color='r')
```


Результат выполнения:



Тексту можно также указать поворот `rotation`, который принимает значения `'vertical'`, `'horizontal'` или угол вращения, указанный в градусах.

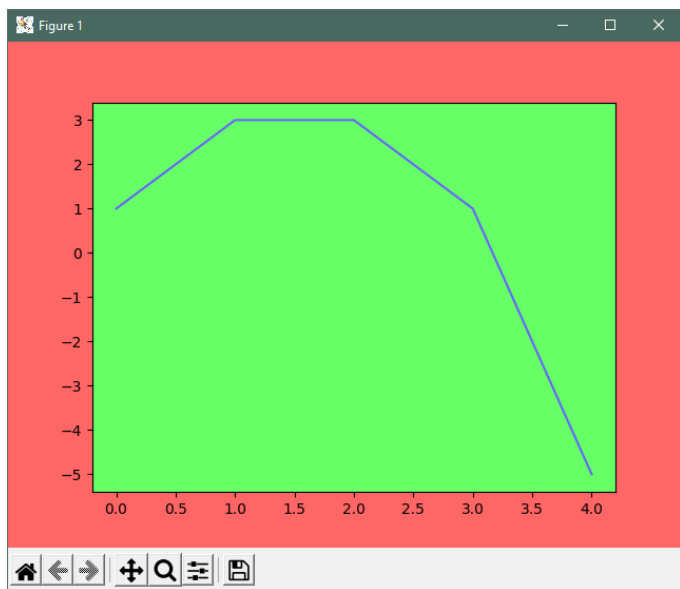
2.3. figure и axes

`Figure` — это контейнер самого верхнего уровня. Таких контейнеров может быть несколько, каждый из которых может содержать несколько контейнеров `Axes`.

`Axes` — это область на которой отображаются графики, а также все вспомогательные атрибуты (линии сетки, метки, указатели и т.д.):

```
1 | fig = p.figure()
2 | fig.patch.set_facecolor('#f66') # задаем цвет для
   | figure (красноватый)
3 | ax = p.axes()
4 | ax.set_facecolor('#6f6') # задаем цвет для axes
   | (зеленый)
5 | p.plot(x, y1, color = '#66f')
```

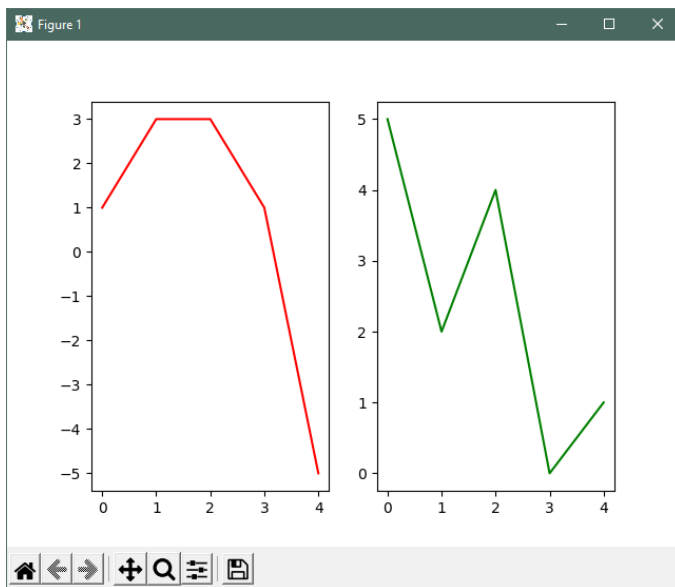
Для понимания что есть что, в коде были заданы различные цвета для областей figure и axes. Результат выполнения данного кода:



Чтобы разместить несколько графиков рядом, надо создать несколько областей Axes, например, следующим образом:

```
1 | fig = p.figure()
2 |
3 | ax0 = fig.add_subplot(121)
4 | p.plot(x, y1, color = 'r')
5 |
6 | ax1 = fig.add_subplot(122)
7 | p.plot(x, y2, color = 'g')
```

В результате, два графика окажутся рядом, каждый с отдельными осями X и Y:



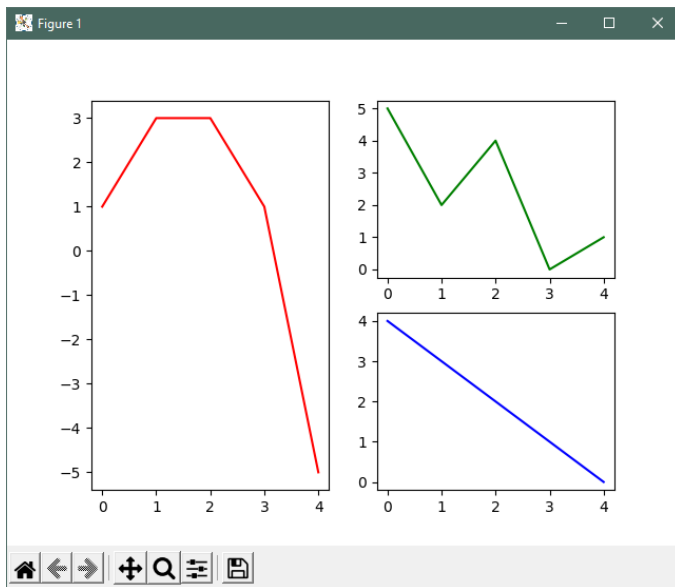
При этом параметр метода `add_subplot` состоит из трех цифр:

- количество строк,
- количество столбцов,
- порядковый номер при таком разбиении.

Более сложный пример:

```
1 | ax0 = fig.add_subplot(121) # На 2 столбца
2 | ax1 = fig.add_subplot(222) # 2 столб. 2 стр.
3 | ax2 = fig.add_subplot(224) # Аналогично
```

Результат:



Здесь ax0 это красный график, так как figure делилась только на два столбца и не делилась на строки (параметр начинается с 12, то есть одна строка и два столбца), график занимает всю левую половину. Для следующих двух графиков figure делилась на четверти (параметр начинается с 22).

Для каждой из областей рисования можно задать свои подписи и границы обзора:

```
1 | ax0 = fig.add_subplot(121) # График слева
2 | ax0.set_xlim([-1, 5]) # Ограничение видимой
  | области
3 | ax0.set_ylim([0, 5])
4 | p.plot(x, y2, color = 'r')
5 |
6 | ax1 = fig.add_subplot(122) # График справа
7 | ax1.set_xlim([-3, 15])
8 | ax1.set_ylim([-3, 15])
9 | ax1.set_title('Область [-3; 15]') # Подпись к
  | графику
10 | ax1.set_xlabel('X') # Подписи к осям
```

```

11 | ax1.set_ylabel('Y')
12 | p.plot(x, y2, color = 'r')

```

В результате будет нарисована одна и та же функция, но с различным ограничением области отображения. На правом графике также будут присутствовать подпись и названия осей:



Изменить **название окна**, можно передав строку в figure:

```

1 | fig = p.figure('Мой заголовок')

```

Размер окна в дюймах также указывается как параметр figsize метода figure:

```

1 | fig = p.figure(figsize=(5, 2.5))

```

2.4. Полярная система координат

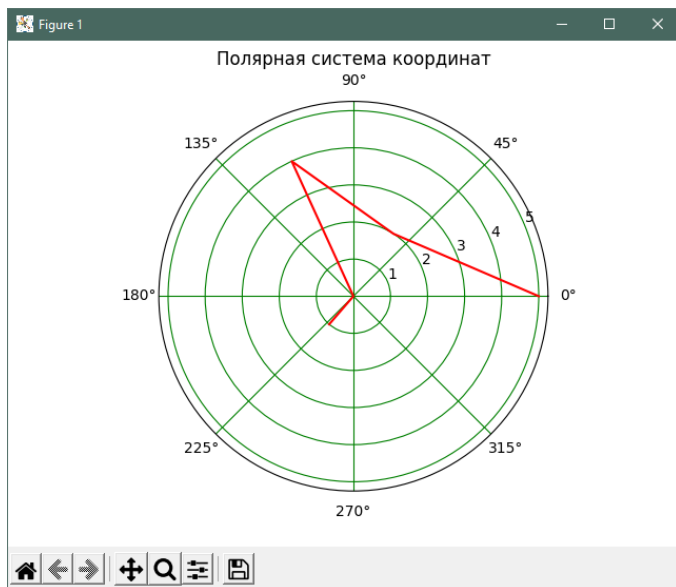
Графики можно строить в полярной системе координат, указав параметр polar как True:

```

1 | ax1 = fig.add_subplot(111, polar=True)
2 | ax1.set_title('Полярная система координат') #
  | Подпись
3 | ax1.grid(visible=True, color='g') # Сетка
4 | p.plot(x, y2, color = 'r')

```

Результат:



2.5. Графики с numpy

Для построения более сложных графиков может потребоваться подключить библиотеку numpy:

```
1 | import matplotlib.pyplot as p
2 | import numpy as np
```

Функция `numpy.arange()` библиотеки NumPy возвращает равномерно расположенные значения внутри заданного интервала с указанным шагом

Отличие от `range`:

- возвращает NumPy-массив, а не список,
- шаг не обязан быть целым числом.

Массивы в NumPy заточены под скорость, а также под удобство выполнения операций над ними. Например, можно увеличить все элементы на значение простым сложением:

```
1 | arr = np.arange(0, 5, 1)
2 | # [0 1 2 3 4]
3 |
```

```
4 | arr = arr + 10
5 | # [10 11 12 13 14]
```

Множество функций библиотеки (некоторые пересекаются с функциями `math`) также ориентированы на скорость и работу с массивами:

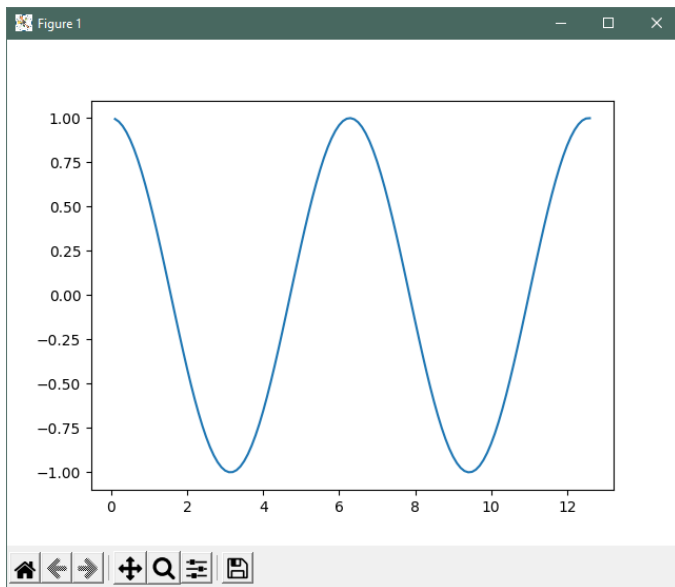
- тригонометрические (`sin`, `cos`, `tan`, `arccos`, `arcsin`, `arctan`),
- квадратный корень (`sqrt`) и степень (`power`),
- модуль (`abs`),
- экспонента (`exp`) и логарифм (`log`, `log2`, `log10`),
- сумма (`sum`) и среднее значение (`mean`),
- максимальное (`max`) и минимальное (`min`) значения,
- медиана, дисперсия, среднеквадратическое отклонение...

Также поддерживается работа с матрицами.

Следующий код демонстрирует использование библиотеки для построения графика функции косинуса:

```
1 | x = np.arange(0.1, 4*np.pi+0.1, 0.1)
2 | y = np.cos(x)
3 | p.plot(x, y)
4 | p.show()
```

Результат построения:



2.6. Геометрические объекты

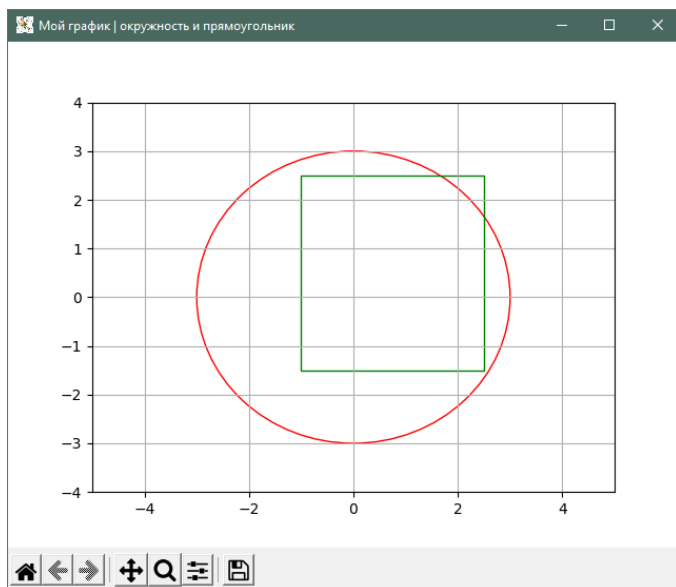
Задание круга и прямоугольника (центр, радиус, цвет, fill – залить или только контур):

```
1 | c1 = p.Circle((0, 0), 3, color='r', fill=False)
2 | r1 = p.Rectangle((-1, -1.5), 3.5, 4, color='g',
   | fill=False)
```

Для их построения потребуется метод gca:

```
3 | ax0 = fig.add_subplot(111)
4 | ax0.set_xlim([-5, 5])
5 | ax0.set_ylim([-4, 4])
6 | ax0.grid(visible=True)
7 |
8 | ax2 = p.gca() # Конва (из текущих осей)
9 | ax2.add_patch(c1) # Добавление круга
10 | ax2.add_patch(r1) # Добавление прямоугольника
11 | p.show()
```


Результат построения:



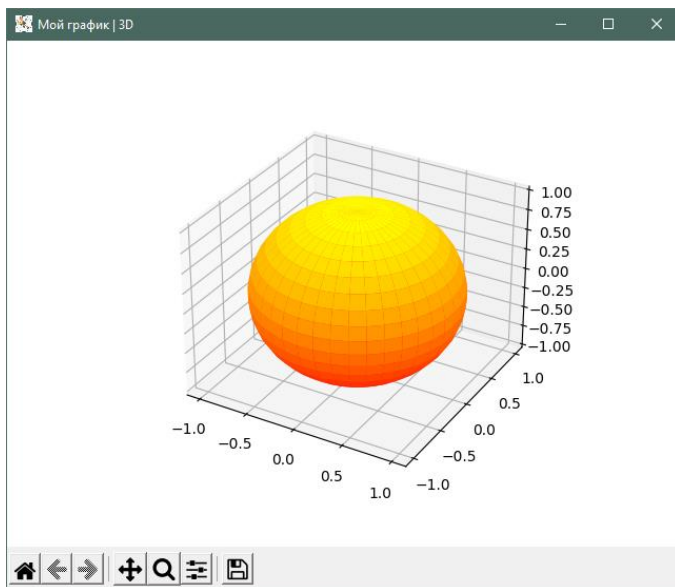
2.7. Трехмерные графики

Чтобы указать, что график трехмерный, нужно сделать следующее действие:

```
1 | ax = fig.add_subplot(111, projection='3d')
```

На вход функции `plot` придется указать уже не 2 массива, а 3 матрицы, и не цвет, а цветовую карту.

Пример графика в трех координатах:



3. Задачи на matplotlib

3.1. Пример решения

Задача 1. Построить график функции $y = \sqrt[3]{x^2} \pm \sqrt{1 - x^2}$.

Решение:

```
import matplotlib.pyplot as p
import numpy as np
```

```
# Настройки отображения
```

```
fig = p.figure('♥ Мой график ♥') # Название окна
```

```
thickness = 5 # Толщина линий (используется ниже)
```

```
line_color = 'red'
```

```
# Параметры
```

```
beg = -1
```

```
end = 1
```

```
steep = 0.01
```

```

# Генерация массива "по X"
x = np.arange(beg, end, steep)
x = np.append(x, end) # Добавление правой границы

# Массивы Y
y1 = pow(x**2, 1/3) + pow(1-x**2, 1/2)
y2 = pow(x**2, 1/3) - pow(1-x**2, 1/2)

# Отрисовка
p.plot(x, y1, linewidth=thickness, color=line_color)
p.plot(x, y2, linewidth=thickness, color=line_color)
p.show()

```

Описание.

Вначале подключаются библиотеки `matplotlib` и `numpy` и задаются псевдонимы `p` и `np` соответственно.

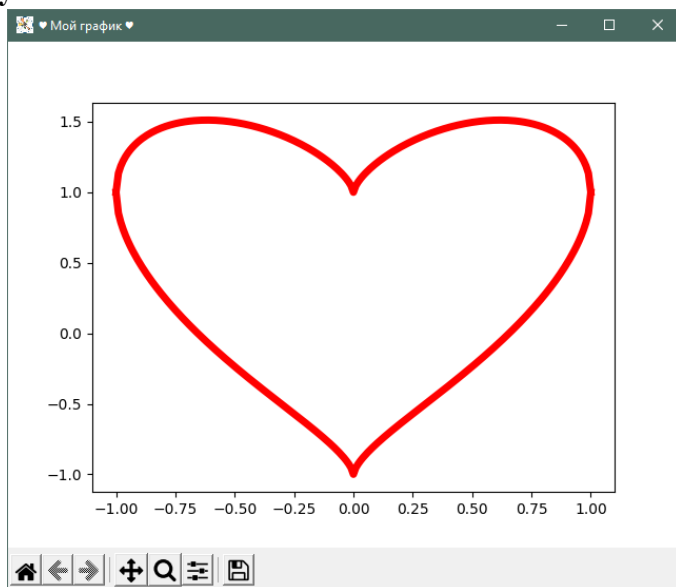
Далее настраивается заголовок окна, толщина и цвет линий сохраняются в переменные.

Для удобства редактирования кода переменные, отвечающие за начало, конец и шаг рассматриваемых вдоль оси X значений сгруппированы в одном месте.

После этого, функцией `arange` генерируется массив значений x .

Два массива $y1$ и $y2$ вычисляются по формуле из условия задачи. После чего вызываются методы `plot` с соответствующими наборами абсцисс, ординат, толщиной и цветом линии. В конце вызывается `show` для отображения всех графиков.

Результат:



Задача 2. Построить в полярных координатах график функции $r=2*t$.

Решение:

```
import matplotlib.pyplot as p
import numpy as np
```

```
# Настройки вида
```

```
fig = p.figure('Мой график | полярная система')
line_color = '#345247'
```

```
# Параметры
```

```
steep = np.pi / 25
laps = 3
```

```
# Массивы
```

```
t = np.arange(0, laps*2*np.pi+steep, steep)
r = t*2
```

Отрисовка

```
ax = fig.add_subplot(111, polar=True)
p.plot(t, r, color=line_color, linewidth=2)
p.show()
```

Описание.

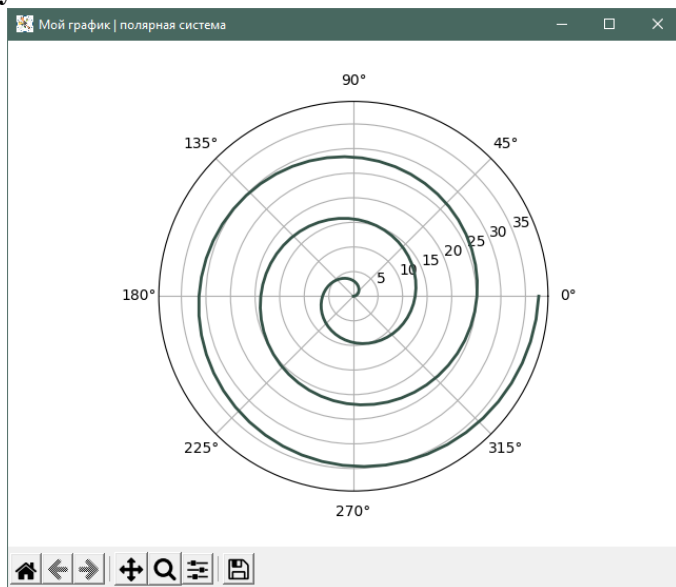
Аналогично предыдущей задаче, вначале подключаются библиотеки, настраивается заголовок окна, толщина и цвет линий.

Параметры, определяющие шаг изменения угла t и количество полных кругов (2π) вынесены перед остальным кодом для удобства редактирования.

После этого, функцией `arange` генерируется массив значений t , и массив r вычисляется по формуле.

Далее вызывается методы `plot` и `show` для отображения всех графика.

Результат:



3.2. Условия

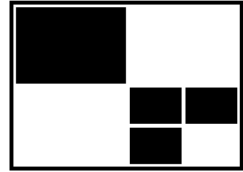
Задача должна быть выполнена на языке Python с использованием библиотек matplotlib, numpy, math и/или random. Условие задачи и автор в обязательны к указанию в комментариях.

Все графики строятся в одном окне. Название окна должно быть осмысленным и не должно оставаться по умолчанию. Область рисования должна быть настроена так, чтобы графики отображались корректно.

1. Построить 4 графика, расположив их в окне как на схеме.

Толщина линии во всех графиках = 3.

Все графики выполнить в различных оттенках синего цвета.



- $x = \cos 3t \cos t$
 $y = \cos 3t \sin t$, где $t \in [0; 2\pi]$,
- $y = A/x$,
- $y = B * \ln x$,
- треугольник с координатами вершин (C,C),(0,0),(D,E).

Коэффициенты A, B, C, D, E получаются случайным образом из диапазона [0.1; 10].

2. Построить 4 графика, расположив их в окне как на схеме.

Толщина линии во всех графиках = 5.

Все графики выполнить в различных оттенках красного цвета.



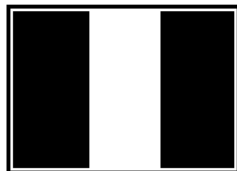
- $x = 6 * \cos t + \cos 6t$
 $y = 6 * \sin t - \sin 6t$, где $t \in [0; 2\pi]$,
- $y = \cos Ax$,
- $y = B^x$,
- ломанную (0,0),(1,C),(2,D),(3,E).

Коэффициенты A, B, C, D, E получаются случайным образом из диапазона [0.1; 10].

3. Построить 4 графика (по два в каждом поле), расположив их в окне как на схеме.

Все графики одного цвета.

В каждом из полей один из графиков представлен сплошной линией, другой — пунктирной.



- $|y| = \frac{\sqrt{2+4x \pm 2\sqrt{1-4x}}}{1 \mp \sqrt{1-4x}}$, где $x \in [-2; 0.25]$,
- $y = Ax^4 + Bx^2 + C$,
- $y = \tan Dx$,
- $y = \cot Ex$.

Коэффициенты A, B, C, D, E получаются случайным образом из диапазона [0.1; 10].

4. Построить 4 графика (по два в каждом поле), расположив их в окне как на схеме.

Все графики одного цвета.

В каждом из полей один из графиков представлен сплошной линией, другой — точечной.



- $y = \pm \sqrt{\sqrt{1+4x^2} - x^2 - 1}$,
- $y = A \sin x + B$,
- $y = Ce^x + D$,
- $y = 100Ex$.

Коэффициенты A, B, C, D, E получаются случайным образом из диапазона [0.1; 10].

5. Построить 2 графика, расположив их в окне как на схеме.

- В декартовой системе: $y = 2^{\frac{x}{10}} \cos x$,
- В полярной системе: $r = \sin \frac{111\varphi}{122}$, где $\varphi \in [0; 100\pi]$.

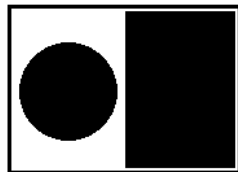


Включить сетку в поле с декартовой системой координат.

6. Построить 2 графика, расположив их в окне как на схеме.

– В декартовой системе: $y = e^{\frac{-x}{15}} \sin x$,

– В полярной системе:
 $r = 100(1.2 + 0.9 \cos(1.8\varphi) + 0.1 \cos(0.9\varphi))$,
где $\varphi \in [0; 20\pi]$.



Убрать в поле с полярной системой координат подписи значений осей.

4. Библиотека NumPy

4.1. Массивы

Библиотека NumPy используется для научно-технических расчетов и высокопроизводительных вычислений. Она предоставляет доступ к оптимизированным математическим операциям над одномерными и многомерными массивами, средства для ввода и вывода данных и другие средства для проведения научных расчетов.

В пакете предоставляется новый тип данных — ndarray (n-мерный массив), над которым возможно выполнение векторных операций посредством одной команды (без явного использования циклов).

Для подключения модуля используется следующая команда:

```
1 | import numpy as np
```

Следующие отличия от стандартных списков Python характерны массивам ndarray:

- массивы однородны (все элементы одного типа),
- массивы имеют фиксированный размер,
- математические операции над элементами массива чаще всего менее ресурсозатратны.

Чтобы создать переменную такого типа, используется ключевое слово **array**:

```
2 | x = np.array([1, 2])
3 | print(x)          # [1 2]
4 | print(type(x))    # <class 'numpy.ndarray'>
```

Чтобы узнать ассоциированный тип (тип хранимых значений), необходимо использовать атрибут **dtype**:

```
5 | print(x.dtype)    # int64
```

Можно узнать следующую информацию о массиве: **ndim** (размерность, то есть число измерений), **size** (длина, то есть суммарное число объектов по всем измерениям), **shape** (форма, то есть кортеж целых чисел, указывающих размер массива по каждому измерению):

```
6 | print(x.ndim)     # 1
7 | print(x.size)     # 2
8 | print(x.shape)    # (2, )
```

Аналогичный пример для матрицы:

```
1 | import numpy as np
2 | y = np.array([[1, 2, 3], [4, 5, 6]])
3 | print(y.ndim)     # 2
4 | print(y.size)     # 6
5 | print(y.shape)    # (2, 3)
```

Чтобы узнать размер в байтах одного элемента массива используется атрибут **itemsize**:

```
6 | print(y.itemsize) # 8
```

Для создания массивов заданной длины, заполненных нулями или единицами, используются методы **zeros** и **ones**:

```
1 | import numpy as np
2 | zr = np.zeros(shape=(5,), dtype=int) # [0 0 0 0 0]
3 | on = np.ones(shape=(5,), dtype=int)  # [1 1 1 1 1]
```

Где параметр `dtype` не обязателен, и, если его не указать, будет иметь значение `float`, а параметр `shape` можно указывать без имени, и, в одномерном случае, число не обязательно помещать в кортеж, например, вызов функции может происходить как: `zr = np.zeros(5)`. В таком случае массив будет иметь вид `[0. 0. 0. 0. 0.]`.

4.2. Операции над массивами

Арифметические и **битовые** операции, а также **сравнения** применяются к массивам `ndarray` поэлементно:

```
1 | import numpy as np
2 | x = np.array([1, 2, 3])
3 | y = np.array([4, 5, 6])
4 | print(x + 10)      # [11 12 13]
5 | print(x + y)       # [5 7 9]
6 | print(x >= 2)      # [False True True]
7 | print(x | 2)       # [3 2 3]
8 | print(x << 2)      # [ 4 8 12]
```

В строке 4 происходит сложение каждого элемента массива с числом 10, в строке 5 — поэлементное сложение двух массивов, в строке 6 формируется массив результатов сравнения для каждого элемента, в строках 7 и 8 выполняются битовые операции над каждым элементом.

Матричное произведение вычисляется методом `dot` или оператором `@`:

```
1 | import numpy as np
2 | x = np.array([[4], [5]])
3 | y = np.array([[1, 2, 3]])
4 | print(x @ y)
```

Результат:

```
[[ 4  8 12]
 [ 5 10 15]]
```

Вычисление **минимума** и **максимума** стандартными функциями Python возможны только для одномерного массива.

Для произвольного числа измерений используются функции `np.maximum` и `np.minimum`.

Для матриц определен метод **транспонирования** «.T»:

```
1 | import numpy as np
2 | x = np.array([[1, 2, 3], [4, 5, 6]])
3 | print(x.T)
```

Следующие методы позволяют вычислить:

- `trace` — след матрицы,
- `linalg.inv` — обратную матрицу,
- `linalg.det` — определитель матрицы,
- `linalg.matrix_rank` — ранг матрицы,
- `linalg.matrix_power` — возведение в целочисленную степень,
- `linalg.eigvals` — собственные значения матрицы,
- `linalg.solve(A, B)` — решение системы линейных уравнений $Ax = B$.

Также поддерживаются следующие унарные функции над массивами: `sqrt`, `exp`, `sin`, `cos`, `log`, `log2` и бинарные функции `add`, `subtract`, `multiply`. Например:

```
1 | import numpy as np
2 | x = np.array([1, 2, 3])
3 | print(np.sqrt(x))
```

Срезы вычисляются по каждой оси отдельно:

```
1 | x = np.array([[1, 2, 3, 4],
2 |               [5, 6, 7, 8],
3 |               [9, 10, 11, 12]])
4 | print(x[1:, 2:4])
```

В результате будет напечатан срез с первой строки до конца и со второго по четвертый (не включительно) столбец, то есть:

```
| [[ 7  8],
|  [11 12]]
```

Индексы по всем измерениям начинаются с нуля.

4.3. Пакет SciPy

Данный пакет предназначен для проведения сложных научно-технических расчетов и содержит модули для решения задач линейной алгебры, анализа сигналов и обработки изображений.

И NumPy, и SciPy являются библиотеками Python, которые используются для математического и числового анализов. NumPy содержит данные массивов и операции, такие как сортировка, индексация, а SciPy состоит из числового кода. И хотя в NumPy есть функции для работы с линейной алгеброй, преобразованиями Фурье и прочие, в SciPy они представлены в полном виде вместе с массой других. А для полноценного научного анализа в Python нужно устанавливать и NumPy, и SciPy, поскольку последняя построена на базе NumPy.

В SciPy есть набор пакетов для разных научных вычислений:

- cluster — алгоритмы кластерного анализа,
- constants — физические и математические константы,
- fftpack — быстрое преобразование Фурье,
- integrate — решения интегральных и обычных дифференциальных уравнений,
- interpolate — интерполяция и сглаживание сплайнов,
- io — ввод и вывод,
- linalg — линейная алгебра,
- ndimage — n-размерная обработка изображений,
- odr — метод ортогональных расстояний,
- optimize — оптимизация и численное решение уравнений,
- signal — обработка сигналов,
- sparse — разреженные матрицы,
- spatial — разреженные структуры данных и алгоритмы,
- special — специальные функции,
- stats — статистические распределения и функции.

5. ООП

5.1. Объектно-ориентированное программирование

Объектно-ориентированное программирование используется для того, чтобы связать поведение сущности с её данными и спроецировать объекты реального мира и бизнес-процессов в программный код.

Класс — это описание того, какими свойствами и поведением будет обладать объект. А объект — это экземпляр с собственным состоянием этих свойств.

Три основных принципа ООП:

1. Наследование — это механизм системы, который позволяет наследовать одними классами свойства и поведение других классов для дальнейшего расширения или модификации, что позволяет избегать лишней работы, дублирования кода и логики.

Также механизм наследования позволяет создавать общий каркас для разных классов, например, квадрат, круг и треугольник — фигуры на плоскости, для каждой из которых может быть вычислена ее периметр и площадь. Для многих решений может оказаться удобным сделать эти три класса производными от общего родительского класса-фигуры с методами нахождения площади и периметра.

2. Полиморфизм — свойство системы, позволяющее иметь множество реализаций одного интерфейса. То есть внешний код ничего не знает о реализации вызываемого метода.

Полиморфизм дословно означает «множество форм» — вызывающий код видит только название функции, не зная и не заботясь о том, какую конкретно форму (тело функции) она принимает. Например, метод вычисления площади для квадрата и круга может выглядеть одинаково, однако квадрат вычисляет площадь как «длина стороны в степени два», а круг — « $\pi \cdot r^2$ ».

3. Инкапсуляция — это контроль доступа к полям и методам объекта. Во многих языках частью инкапсуляции является сокрытие данных (модификаторы доступа). Но под контролем доступа подразумевается не только можно получить доступ или нет,

но и различные валидации, подгрузки, вычисления и другое динамическое поведение.

5.2. Создание класса

Синтаксис для написания нового класса:

```
1 | class ClassName(базовый класс):  
2 |     'Краткое описание класса (необязательно)'  
3 |     # Атрибуты и методы
```

Например:

```
1 | class Point2d(object):  
2 |     'Точка на плоскости'  
3 |     x = 0 # Обязательно присваивается значение  
4 |     y = 0
```

Методы объявляются как простые функции внутри класса, например:

```
1 | class Point2d(object):  
2 |     x = 0  
3 |     y = 0  
4 |     # Длина радиус вектора  
5 |     def lenPV(self):  
6 |         return (self.x**2 + self.y**2) ** 0.5
```

Параметр `self` обязателен и отличает метод класса от обычной функции. Это общепринятое имя для ссылки на объект, в контексте которого вызывается метод. То есть можно придумать другое имя для параметра `self`, но это крайне не приветствуется.

При вызове метода параметр `self` не нужно указывать.

Создание экземпляра класса имеет вид `имя_переменной = имя_класса(параметры):`

```
1 | a = Point2d()  
2 | a.x = 1  
3 | a.y = 1
```

5.3. Специальные (магические) методы

Можно создавать разные экземпляры одного класса с заранее заданными параметрами с помощью **инициализатора** — метода `__init__`:

```
1 | class Point2d(object):
2 |     x = 0
3 |     y = 0
4 |     def __init__(self, x, y):
5 |         self.x, self.y = x, y
6 |
7 | a = Point2d(1, 1)
```

Деструктор имеет вид `__del__`. На практике деструктор используется редко, в основном для тех ресурсов, которые требуют явного освобождения памяти при удалении объекта. Не следует совершать в нем сложные вычисления. Пример:

```
1 | class Point2d(object):
2 |     x = 0
3 |     y = 0
4 |     def __del__(self):
5 |         ... # Деструктор путой
6 |
7 | a = Point2d(1, 1)
8 | del a # Вызов деструктора
```

Оператор `del` вызывался бы без ошибок, даже если не определять метод `__del__`. Этот метод нужен, чтобы при вызове оператора `del`, исполнялся код, написанный в теле метода `__del__`. Например, для дерева (или другой сложной иерархической структуры) полезно вызвать удаление всех ветвей рекурсивно.

Для объектов пользовательского класса можно определить набор других специальных методов.

Так, для создания логики поведения пользовательского объекта как контейнера необходимо определить методы `__len__` (длина), `__getitem__` (получение значения по ключу), `__setitem__` (установка

значения для ключа), `__delitem__`(удаление значения), `__contains__`(проверка наличия).

Определение этих методов позволит использовать экземпляр данного класса как последовательность. Например, обратиться к нему по индексу (с помощью квадратных скобок).

Пользовательские объекты могут участвовать в **математических операциях**, если у них определены специальные методы:

- `__add__` для операции сложения (+),
- `__sub__` для операции вычитания (-),
- `__mul__` для операции умножения (*),
- `__truediv__` для операции деления (/),
- `__floordiv__` для операции целочисленного деления (//),
- `__mod__` для нахождения остатка от деления (%),
- `__pow__` для возведения в степень (**),
- другие.

Для приложения важным может оказаться реализация метода `__str__`, который используется методом `print`, позволяющему выводить информацию на консоль или в файл, а также просто для приведения к строковому типу функцией `str()`.

Метод `__format__` используется функцией `format`, а также методом `format` у строк.

Пример написания пользовательской последовательности:

```
1 | class MyContainer(object):
2 |     name = "
3 |     lst = list()
4 |
5 |     def __init__(self, name, lst):
6 |         self.name = name
7 |         self.lst = lst
8 |
9 |     def __len__(self):
10 |         return len(self.lst)
11 |
12 |     def __getitem__(self, i):
```



```

13         return self.lst[i]
14
15     def __setitem__(self, i, value):
16         self.lst[i] = value
17
18     def __delitem__(self, i):
19         del self.lst[i]
20
21     def __contains__(self, value):
22         return value in self.lst
23
24     def __str__(self):
25         res = self.name + ': '
26         for i in self.lst:
27             res += str(i) + ', '
28         return res[:-2]
29
30     def __del__(self):
31         del self.lst
32
33     # Используется __init__
34     X = MyContainer('Числа', [1, 2, 3])
35     # Используется __setitem__
36     X[0] = 10
37     # Используется __str__
38     print(X)

```

В результате будет напечатано «Числа: 10, 2, 3». Нарезание также поддерживается, но будет возвращать при такой реализации список. То есть `print(X)` выводит «Числа: 10, 2, 3», а `print(X[:])` выводит «[10, 2, 3]».

Для того, чтобы определить **правила для сравнения** двух объектов пользовательского класса используется методы:

- `__eq__` для равенства (`==`),
- `__ne__` для неравенства (`!=`),
- `__lt__` для оператора меньше (`<`),

- `__le__` для оператора меньше или равно (`<=`),
- `__gt__` для оператора больше (`>`),
- `__ge__` для оператора больше или равно (`>=`).

Для определения в каком случае пользовательский объект в условиях будет считаться истинным или ложным используется метод `__bool__`.

Можно определить следующие **битовые операторы**:

- `__lshift__` битовый сдвиг влево (`<<`),
- `__rshift__` битовый сдвиг вправо (`>>`),
- `__and__` битовое И (`&`),
- `__xor__` битовое ИСКЛЮЧАЮЩЕЕ ИЛИ (`^`),
- `__or__` битовое ИЛИ (`|`).

Арифметические и битовые операторы будут использоваться только если экземпляр данного класса являлся левым операндом. Чтобы определить операторы для экземпляров — правых аргументов, (только в случае, если для левого операнда не определён соответствующий метод) перегружаются соответствующие арифметические и битовые операторы с буквой «г» в начале слова, например, `__radd__` и `__rmul__`.

Для сокращенных операций (например, «+=») определяются методы, начинающиеся с префикса «i»: `__iadd__`, `__isub__` и так далее.

Для написания пользовательских **итераторов**, необходимо определить методы `__iter__` и `__next__`. Также можно определить `__reversed__` для итераторов из элементов, следующих в обратном порядке.

Определив метод `__call__` можно вызывать экземпляры класса как функции.

5.4. Инкапсуляция

Все объекты в Python инкапсулируют внутри себя данные и методы работы с ними, предоставляя публичные интерфейсы для взаимодействия.

Можно выделить два подхода к значению этого термина. Инкапсуляция может быть рассмотрена как:

- связь данных с методами, которые этими данными управляют,
- набор инструментов для управления доступом к данным или методам, которые управляют этими данными.

Со вторым пунктом у языка Python проявляются некоторые особенности по сравнению с такими языками как C++ или C#.

Атрибут может быть объявлен приватным (внутренним) с помощью нижнего подчеркивания перед именем, но настоящего скрытия на самом деле не происходит – *все на уровне соглашений*.

```

1 | class Point2d(object):
2 |     x = 0
3 |     y = 0
4 |     # Внутренний метод
5 |     def _hi(self):
6 |         print('Hi, I am point ' + str(self))
7 | a = Point2d()
8 | a._hi() # Вызов все равно возможен

```

В результате выполнится вызов помеченного как закрытый метода `_hi`, который напечатает информацию в консоль.

Двойное подчеркивание поможет скрыть атрибуты класса:

```

1 | class Point2d(object):
2 |     __x = 0
3 |     __y = 0
4 |
5 | a = Point2d()
6 | print(a.__x) # Возникает ошибка
7 | a.__x = 5 # Ошибки нет но значение не изменится

```

Сам язык предоставляет программисту синтаксический инструмент, который может обойти инкапсуляцию. Читать и изменять частные переменные и вызывать частные функции все же возможно. То есть в данном решении есть обходной путь:

```

1 | a = Point2d()
2 | print(a._Point2d__x) # Напечатается ноль

```

```
3 | a._Point2d__x = 5 # Значение изменится
```

Таким образом Python не обеспечивает ограниченный доступ к каким-либо переменным или методам класса. Данные, которые должны быть скрыты, на самом деле могут быть прочитаны и изменены. В Python инкапсуляция является скорее условностью, и программист должен самостоятельно заботиться о ее сохранении.

5.5. Наследование

Все в Python является объектом. Модули — это объекты, определения классов и функции — это объекты, и, конечно, объекты, созданные из классов, тоже являются объектами.

Под наследованием понимается возможность создания нового класса на базе существующего. При этом класс потомок будет содержать те же атрибуты и методы, что и базовый класс, но при этом его можно расширять через добавление новых методов и атрибутов.

Родительский класс помещается в скобки после имени класса:

```
1 | class Point2d(object):  
2 |     x = y = 0  
3 | class Point3d(Point2d):  
4 |     z = 0 # Новое поле
```

Класс `object` в Python является базовым для всех классов. Каждый пользовательский класс в Python явно или неявно происходит от `object` (кроме классов ошибок).

Язык программирования Python реализует как одиночное наследование, так и множественное. Родительские классы при этом указываются через запятую.

5.6. Полиморфизм

Концепция полиморфизма – важная часть ООП на Python. Все методы в языке изначально виртуальные. Это значит, что дочерние классы могут их переопределять и решать одну и ту же задачу разными путями, а конкретная реализация будет выбрана только во время исполнения программы.

```

1 | class C1:
2 |     def Show(self):
3 |         print('C1', end = ' ')
4 |
5 | class C2(C1):
6 |     def Show(self):
7 |         print('C2', end = ' ')
8 |
9 | c = (C1(), C2()) # Кортеж с объектами типа C1 и C2
10 | for i in c:
11 |     i.Show() # Вывод: C1 C2

```

Можно получить и доступ к методам класса-предка либо по прямому обращению, либо с помощью функции `super()`:

```

1 | class C2(C1):
2 |     def Show(self):
3 |         super().Show()
4 |         print('C2', end = ' ')

```

Функция `super()`, возвращает объект-посредник, который делегирует вызовы метода родительскому или родственному классу, указанного `type` типа. Это полезно для доступа к унаследованным методам, которые были переопределены в классе. Необязательным параметром метода является тип, от которого начинается поиск объекта-посредника, что позволяет решить проблему выбора родительского метода при множественном наследовании.

Альтернативный вариант — вызвать нужную функцию напрямую по имени класса:

```

1 | class C2(C1):
2 |     def Show(self):
3 |         C1.Show(self)
4 |         print('C2', end = ' ')

```

5.7. Абстрактные классы

Абстрактный класс — очень важная концепция объектно-ориентированного программирования. Такой класс имеет следующие особенности:

1. Абстрактный класс не содержит всех реализаций методов, необходимых для полной работы, это означает, что он содержит один или несколько абстрактных методов. Абстрактный метод — это только объявление метода, без его подробной реализации.

2. Абстрактный класс предоставляет интерфейс для подклассов, чтобы избежать дублирования кода. Нет смысла создавать экземпляр абстрактного класса.

3. Производный подкласс должен реализовать абстрактные методы для создания конкретного класса, который соответствует интерфейсу, определенному абстрактным классом. Следовательно, экземпляр не может быть создан, пока не будут переопределены все его абстрактные методы.

В стандартную библиотеку Python включается модуль `abc`, добавляющий в язык абстрактные базовые классы:

```
1 | from abc import ABC, abstractmethod
```

Абстрактный класс можно определить с помощью наследования от класса `ABC`:

```
2 | class MyBase(ABC):
```

Таким образом можно определить класс, указав при этом, какие методы или свойства обязательно переопределить в классах-наследниках:

```
3 |     @abstractmethod # См. "Декораторы"
4 |     def Show(self):
5 |         pass
```

Абстрактный класс не может быть унаследован, пока не будут переопределены все его абстрактные методы и свойства:

```
1 | from abc import ABC, abstractmethod
2 |
3 | class MyBase(ABC):
4 |     @abstractmethod
5 |     def Show(self):
6 |         pass
7 |
```

```

8 | class MyMain(MyBase):
9 |     pass
10 |
11 | m = MyMain()

```

В ходе выполнения данного кода возникнет ошибка: нельзя создать экземпляр, не переопределив все абстрактные методы методы.

Абстрактный метод в Python не обязательно должен быть «полностью абстрактным», то есть в нем может содержаться некоторый код, и использоваться наследниками с помощью `super`.

5.8. Статические методы и методы класса

Для создания **статических методов** в Python предназначен декоратор `@staticmethod`. У них нет параметра `self`. Статический метод не может изменять ни состояние объекта, ни состояние класса. Он ограничен в том, к каким данным может получить доступ.

Важно, что в Python можно вызвать статический метод не только по имени класса, но и для экземпляра объекта. Экземпляр в таком случае просто не передается:

```

1 | class MyClass():
2 |     @staticmethod
3 |     def Show():
4 |         print('MyClass')
5 |
6 | m = MyClass()
7 | m.Show() # Так можно
8 |
9 | MyClass.Show() # И так можно

```

Методы класса принимают класс в качестве параметра, который принято обозначать как `cls`. Он указывает на класс, а не на объект этого класса. При объявлении методов этого вида используется декоратор `@classmethod`.

Методы класса привязаны к самому классу, а не его экземпляру. Они могут менять состояние класса, что отразится

на всех объектах этого класса, но не могут менять конкретный объект.

Такие методы позволяют создать альтернативный конструктор или несколько конструкторов класса.

5.9. Статические свойства/поля

Статических свойств, которые можно было бы сделать по аналогии со статическими методами в Python нет, но есть возможность обращаться ко всем полям по имени класса и тогда они будут использоваться как статические, например:

```
1 | class Ant:
2 |     TOTAL = 0
3 |     ant_type = "
4 |
5 |     def __init__(self, ant_type):
6 |         self.ant_type = ant_type
7 |         Ant.TOTAL += 1 # Статическое
8 |
9 | w = Ant('Worker')
10 | q = Ant('Queen')
11 | print(Ant.TOTAL) # 2
```

6. Аннотации

Python — язык с динамической типизацией, что позволяет свободно оперировать переменными разных типов. Однако, при написании кода, типы переменных, которые будут использоваться, всегда предполагаются.

Сохраняя идею динамической типизации, в современных версиях Python поддерживаются аннотации:

- типов переменных,
- полей класса,
- аргументов и возвращаемых значений функций.

Аннотации типов позволяют добавлять подсказки о типах переменных. Они используются, чтобы информировать читателя кода, каким должен быть тип переменной. Это придаёт немного статический вид коду на динамически типизированном Python.

Аннотации типов просто считываются интерпретатором Python и никак более не обрабатываются.

Аннотации типов поддерживаются многими IDE для Python, которые выделяют некорректный код или выдают подсказки в процессе набора текста.

Синтаксис аннотаций: «переменная: тип = значение».

Например:

```
1 | x: int = 25
2 | y: int = 'hi' # Ошибки не возникает
3 | print(x, y)
```

Используя аннотации типов, можно указать какие параметры ожидает пользовательская функция:

```
1 | def mult(a: float, b: float): # Аннотации для a и b
2 |     return a*b;
3 |
4 | print(mult(2., 4.))
5 | print(mult(2, 4)) # Ошибки не возникает
6 | print(mult('hi', 4)) # Ошибки не возникает
```

Подобным образом можно указать и ожидаемый тип возвращаемого значения функции с помощью «->»:

```
1 | def mult(a: float, b: float) -> float:
2 |     return a*b;
```

Для использования не примитивных типов (а таких как Dict, Tuple, List, Set) потребуется импортировать модуль typing, например:

```
1 | from typing import List
```

Если пользовательская функция возвращает несколько значений в виде кортежа, ожидаемый результат аннотируется следующим образом: typing.Tuple[<тип 1>, <тип 2>, ...].

Ключевое слово typing.Optional[тип] означает, что переменная будет указанного типа либо None.

Ключевое слово `typing.Union[тип1, тип2...]` означает, что переменная может иметь любой из перечисленных типов.

```
1 | from typing import Union
2 |
3 | def mult(a: Union[float, str], b: float) -> float:
4 |     return a*b;
```

Аннотация `Any` используется если нет нужды ограничивать возможные типы переменной. Например, если это действительно не важно, или в случае самостоятельной обработки типов.

7. Исключения

7.1. Обработка исключений

Исключения — это сценарии ошибок, которые изменяют нормальный поток выполнения программы. Процесс обработки возможных исключений называется обработкой исключений.

Если исключения не обрабатываются должным образом, программа может завершиться преждевременно. Это может привести к повреждению данных или нежелательным результатам.

Обработка исключений Python выполняется с помощью следующего набора ключевых слов:

1. Блок **try** содержит код, который может вызывать исключения или ошибки.

2. Блок **except** используется для перехвата исключений и их обработки. Код блока `except` выполняется только при возникновении соответствующего исключения. Может быть несколько блоков `except`, а также один `except` может обрабатывать несколько исключений.

3. Код блока **finally** всегда выполняется независимо от того, правильно ли выполнялась программа или возникла исключительная ситуация.

4. Код внутри блока **else** выполняется, если не возникают исключения.

Пример использования:

```

1 | a = int(input('Введите делимое '))
2 | b = int(input('Введите делитель '))
3 | try:
4 |     x = a / b
5 | except ZeroDivisionError as e:
6 |     print('Ошибка:', e)
7 | else:
8 |     print(x)

```

В данном случае, если пользователь введет делитель равный нулю, то получит информацию об ошибке, иначе — результат деления.

Ключевое слово **as** — сохраняет ошибку в переменную.

Некоторые из встроенных классов исключений в Python:

- `ArithmeticError` — базовый класс для арифметических ошибок,
- `AssertionError` — возникает, когда утверждение не выполняется,
- `AttributeError` — атрибут не найден,
- `BufferError`,
- `EOFError` — чтение после конца файла,
- `ImportError` — импортированный модуль не найден,
- `LookupError` — базовое исключение для ошибок поиска,
- `MemoryError` — при нехватке памяти,
- `NameError` — когда имя не найдено глобально,
- `OSError` — базовый класс для ошибок ввода-вывода,
- `ReferenceError`,
- `StopIteration`,
- `StopAsyncIteration`,
- `SyntaxError` — недопустимый синтаксис,
- `SystemError` — внутренняя ошибка интерпретатора Python,
- `TypeError` — недопустимый тип аргумента,
- `ValueError` — недопустимое значение аргумента.

Встроенные классы предупреждений:

- класс `Warning` — базовый класс для всех предупреждений,

- BytesWarning — предупреждения, связанные с байтами и буфером, в основном связанные с преобразованием и сравнением строк,
- DeprecationWarning — предупреждение об устаревших функциях,
- FutureWarning — базовый класс для предупреждения о конструкциях, которые будут семантически изменяться в будущем,
- ImportErrorWarning — предупреждение об ошибках при импорте модулей
- PendingDeprecationWarning — предупреждение о функциях, поддержка которых в будущем будет прекращена,
- ResourceWarning — предупреждения об использовании ресурсов
- RuntimeWarning — предупреждения о сомнительном поведении во время выполнения,
- SyntaxWarning — предупреждение о сомнительном синтаксисе,
- UnicodeWarning — предупреждения, связанные с преобразованием Unicode,
- UserWarning — предупреждения, генерируемые кодом пользователя

7.2. Вызов исключений

Исключения можно генерировать самостоятельно — для этого нужно запустить оператор `raise`:

```

1 | def int_sum(a, b):
2 |     if type(a) != int or type(b) != int:
3 |         raise ValueError
4 |     return a+b
5 |
6 | try:
7 |     print(int_sum(1, 5))
8 | except ValueError:
9 |     print('Моя ошибка')
```

Данный пример выведет число 6. Но если передать в функцию `int_sum` не целые числа, то будет напечатано «Моя ошибка».

7.3. Пользовательские исключения

В Python можно создавать свои исключения. При этом есть одно обязательное условие: они должны быть потомками класса `Exception`:

```
1 | class MyError(Exception):  
2 |     def __init__(self, text):  
3 |         self.txt = text  
4 |  
5 | raise MyError('Пользовательская ошибка')
```

При создании своего класса исключения рекомендуется добавлять к его имени суффикс «Error».

8. Декораторы

8.1. Пользовательские декораторы

Декораторы — это обертки вокруг Python-функций (или классов), которые изменяют работу того, к чему они применяются.

Декораторы используются, чтобы изменять работу существующих функций или классов, добавлять новые возможности и обеспечивать безопасность.

Функции в Python являются объектами, соответственно, их можно возвращать из другой функции или передавать в качестве аргумента. Также следует помнить, что функция может быть определена и внутри другой функции.

Ниже представлен пример написания декоратора.

```
1 | # Декоратор, который добавляет скобки  
2 | def MyDecorator(func_to_decor):  
3 |     # Внутри нужно определить функцию-обертку  
4 |     # Она будет подставляться вместо декорируемой  
5 |     # Она вызывает внутри исходную функцию  
6 |     def AddBrackets(x):
```

```

7 |         return '[' + func_to_decor(x) + ']'
8 |     # Нужно вернуть эту функцию
9 |     return AddBrackets
10 |
11 | # Самая обычная функция (закомментирована)
12 | #def MyToString(x):
13 | #     return str(x)
14 |
15 | # 'Применяем' на нее декоратор
16 | @MyDecorator
17 | def MyToString(x):
18 |     return str(x)
19 |
20 | # Вызов функции
21 | print(MyToString(100))

```

Вначале (строки 1-9) описан декоратор, который представляет из себя функцию, которая принимает функцию в качестве аргумента и возвращает также функцию. Функция-аргумент `func_to_decor` — это та функция, которая декорируется. Функция-результат будет использоваться вместо нее во внешнем коде, если применить данный декоратор.

Внутри этой функции объявлена другая (строки 6-7). Суть этой функции такова: сначала выполняется какой-либо дополнительный код, затем вызывается декорируемая функция, так как ее логика не заменяется, а дополняется, после также выполняется код с дополнительной логикой (его может не быть как в начале, так и в конце). В данном примере функция возвращает склейку из квадратных скобок и результата декорируемой функции `func_to_decor`.

Далее (строки 11-13) в комментарии описано как выглядит простейшая функция `MyToString`. Если для данной функции выполнить `print(MyToString(100))`, результатом будет вывод «100».

Однако, прямо перед объявлением функции в строке 16 укажем ранее созданный декоратор `MyDecorator` с помощью символа

«собака». То есть вместо `MyToString` будет подставлена функция, которая добавляет скобки, а уже внутри себя вызывает `MyToString`.

Теперь результатом работы будет вывод «[100]».

Аналогичного результата можно было достигнуть, не используя синтаксис с «@», а явно изменив функцию:

```
1 | # Функция без @
2 | def MyToString(x):
3 |     return str(x)
4 |
5 | MyToString = MyDecorator(MyToString)
6 |
7 | # Вызов функции
8 | print(MyToString(100))
```

В строке 5, аналогично прошлому примеру, вместо функции `MyToString` подставляется результат функции `MyDecorator` — то есть некоторая функция, которая добавляет скобки, а уже внутри себя вызывает `MyToString`.

8.2. Применение декораторов

Можно использовать несколько декораторов для одной функции. При этом важен порядок декорирования.

Можно создавать декораторы для методов классов точно так же, как и для функций, просто не забывая про параметр `self`.

Декораторы несколько замедляют вызов функции.

Невозможно «раздекорировать» функцию. Если функция декорирована — это не отменить.

Декораторы оборачивают функции, что может затруднить отладку.

Декораторы могут быть использованы для расширения возможностей функций из сторонних библиотек (код которых программист не может изменять), или для упрощения отладки.

Также полезно использовать декораторы для расширения различных функций одним и тем же кодом, без повторного его переписывания каждый раз, например:

```

1 | # Декоратор, выводящий время выполнения функции
2 | def FuncTime(func):
3 |     import time as T
4 |     def wrapper(x):
5 |         t = T.time()
6 |         res = func(x)
7 |         print(['I' + func.__name__ + ']: %0.8f) % (T.time() - t))
8 |         return res
9 |     return wrapper
10 |
11 | @ FuncTime
12 | def MyPow1000(x):
13 |     return x ** 1000
14 |
15 | # Вызов функции
16 | print(MyPow1000(2))

```

Такой декоратор может быть применен ко множеству функций, чтобы оценить время их работы.

Чтобы декоратор не зависел от наличия и числа аргументов, можно использовать аргументы `*args` и `**kwargs` следующим образом:

```

1 | # Декоратор, выводящий время выполнения функции
2 | def PerformanceTime(func):
3 |     import time as T
4 |     def wrapper(*args, **kwargs):
5 |         t = T.time()
6 |         res = func(*args, **kwargs)
7 |         print(['I' + func.__name__ + ']: %0.8f) % (T.time() - t))
8 |         return res
9 |     return wrapper

```

В отличие от предыдущего примера, где декоратор мог быть применен только к функции от одного аргумента, данный декоратор может применяться для функций с произвольным числом аргументов, в том числе и без аргументов.

8.3. Встроенные декораторы

Python содержит несколько встроенных декораторов.

Декоратор **@staticmethod** — используется для создания метода, который ничего не знает о классе или экземпляре, через который он был вызван. Поэтому метод можно вызывать без создания экземпляра класса. Такой (статический) метод не принимает `self` в качестве первого аргумента:

```
1 | class Calc:
2 |     @staticmethod
3 |     def Sum(x, y):
4 |         return x + y
5 |
6 | print(Calc.Sum(2, 2)) # 4
```

В последней строке происходит вызов без создания экземпляра класса `Calc`, а просто по названию этого класса. Однако, вызов метода из экземпляра также остается возможным.

Декоратор **@classmethod** — получает класс в качестве неявного первого аргумента (`cls`), точно так же, как обычный метод экземпляра получает экземпляр (`self`). В нем можно использовать свойства класса без создания конкретного экземпляра класса. Такой метод также можно вызывать без создания экземпляра класса.

Декоратор используется, когда нужно получить методы, не относящиеся к какому-либо конкретному экземпляру, но тем не менее, каким-то образом привязанные к классу. Также он может быть переопределен в дочерних классах. Пример:

```
1 | class Printer:
2 |     @classmethod
3 |     def WhoAmI(cls):
4 |         return cls.__name__
5 |
6 | class NewPrinter(Printer):
7 |     pass
8 |
9 | print(Printer.WhoAmI())
```

```
10 | print(NewPrinter.WhoAmI())
```

Метод `WhoAmI` будет выводить название класса, то есть в строке 9 будет напечатано «Printer», а в строке 10 — «NewPrinter».

Декоратор **@property** превращает метод в свойство. К нему можно будет обращаться через точку без скобочек:

```
1 | class MyList:
2 |     __lst = None
3 |     def __init__(self, lst):
4 |         self.__lst = lst
5 |
6 |     @property
7 |     def Sum(self):
8 |         return sum(self.__lst)
9 |
10 | x = MyList([4, 8, 15, 16, 23, 42])
11 | print(x.Sum) # 108
```

В строке 11 происходит обращение к `Sum` как к полю класса `MyList`. Значение при этом будет вычисляться. Однако попытка присваивания (`x.Sum = 50`) вызовет ошибку. Таким образом данный декоратор позволяет создать *геттеры*.

Декоратор **@имя.setter** позволяет написать *сеттеры*. Пример:

```
1 | class Man:
2 |     __name = None
3 |     def __init__(self, name = None):
4 |         self.__name = name
5 |
6 |     @property # геттер
7 |     def name(self):
8 |         return self.__name
9 |
10 |    @name.setter # сеттер
11 |    def name(self, value):
12 |        self.__name = value
13 |
```

```

14 | x = Man()
15 | x.name = 'Андрей'
16 | print(x.name) # Андрей

```

Геттеры и сеттеры позволяют добавить более сложную логику, чем прямой доступ к полю. Например, можно было автоматически бы приводить имя человека к нужному регистру (с заглавной буквы) при присваивании, изменив тело метода в 10-12 строк. При прямом доступе (без геттеров и сеттеров), приходилось бы делать это вручную во внешнем вызывающем коде.

8.4. Декораторы классов

Декоратор класса принимает на вход класс (объект с типом `type` в Python) и возвращает модифицированный класс:

```

1 | import time
2 |
3 | def MyClassDec(cls):
4 |     class Wrapper:
5 |         # поле будет хранить время создания
6 |         __cr_time = None
7 |
8 |         # изменение конструктора
9 |         def __init__(self, *args):
10 |             # wrapped хранит исходный класс
11 |             self.wrapped = cls(*args)
12 |             # сохранение времени
13 |             self.__cr_time = time.ctime(time.time())
14 |
15 |         # обертка приведения к строке
16 |         def __str__(self):
17 |             str_cl = str(self.wrapped)
18 |             str_tm = str(self.__cr_time)
19 |             return str_cl + ' создано [' + str_tm + ']'
20 |     return Wrapper
21 |
22 | @ MyClassDec # класс декорируется

```

```

23 | class Point:
24 |     __x = 0
25 |     __y = 0
26 |     def __init__(self, x, y):
27 |         self.__x = x
28 |         self.__y = y
29 |
30 |     def __str__(self):
31 |         return '(%d; %d)' % (self.__x, self.__y)
32 |
33 | c = Point(1, 1)
34 | print(c)

```

Результат печати «(1; 1) создано [Thu Dec 8 11:50:41 2022]».

8.5. Декораторы с аргументами

В декоратор можно передавать аргументы. Однако декоратор должен принять функцию в качестве первого аргумента, поэтому потребуется дополнительный уровень вложенности.

Пример — декоратор, повторяющий функцию N раз:

```

1 | def repeat(N):
2 |     def repeat_dec(func):
3 |         def wrapper(*args, **kwargs):
4 |             for i in range(N):
5 |                 func(*args, **kwargs)
6 |             return wrapper
7 |         return repeat_dec
8 |
9 | @repeat(3)
10 | def HelloWorld():
11 |     print("Hello World")
12 |
13 | HelloWorld()

```

В результате «Hello World» будет напечатано трижды. Тут `repeat(3)` возвращает декоратор `repeat_dec`, который в свою очередь подменяет функцию, на функцию `wrapper`.

9. Комплексные числа

Комплексное число — это любое число в форме $a + bj$, где a и b — действительные числа, а $j*j = -1$.

В Python есть несколько способов создать такое комплексное число: можно напрямую использовать синтаксис $a + bj$ для создания комплексного числа, либо использовать класс `complex`:

```
1 | x = 5 + 2j
2 | y = complex(5, 2)
```

Чтобы получить значения действительной (**real**) и мнимой (**imag**) частей комплексного числа, используется синтаксис вызова через точку:

```
1 | print(x.real) # 5.0
2 | print(x.imag) # 2.0
```

Сопряжение находится методом **conjugate**.

Комплексные числа поддерживают арифметические операции: их можно складывать, вычитать, умножать и делить как друг на друга, так и на вещественные числа:

```
1 | x = 5 + 2j
2 | y = complex(5, 2)
3 |
4 | print(x * y) # (21+20j)
5 | print(x * 2) # (10+4j)
```

В отличие от действительных чисел, невозможно сравнивать два комплексных числа. Можно сравнивать только их действительную и мнимую части по отдельности.

10. Длинная арифметика

При работе с целыми числами в Python, не может произойти переполнение типов, так как в языке используется длинная

арифметика. Представления чисел ограничены только доступной памятью системы.

Пример:

```
1 | print(2**1000)
```

Результат:

```
1071508607186267320948425049060001810561404811705533607443
7503883703510511249361224931983788156958581275946729175531
4682518714528569231404359845775746985748039345677748242309
8542107460506237114187795418215304647498358194126739876755
9165543946077062914571196477686542167660429831652624386837
205668069376
```

Реализовано это следующим образом: число хранится в виде массива, каждый элемент этого массива — числа в системе счисления с основанием 2^{30} (1.073.741.824).

Выполнение операций сложения, вычитания и умножения, при представлении числа в виде массива, основано на операциях «в столбик». Деление и другие операции над целыми числами имеют более сложную логику.

В случаях, использования не встроенных целых чисел, а библиотек `numpy`, `scipy`, `pandas` и других, может произойти переполнение при работе с целыми числами, так как структуры, лежащие в основе этих библиотек, для более эффективного использования памяти, полагаются на соответствующие C-типы ограниченной точности.

11. Множественный выбор `match`

Начиная с версии языка Python 3.10, оператор `match-case` призван заменить `switch-case` языка СИ.

Он имеет следующий синтаксис:

```
1 | match x:
2 |     case состояние_1:
3 |         блок_кода_1
4 |     case состояние_2:
5 |         блок_кода_2
```

```

6 | case _:
7 |     блок_кода_3

```

После выполнения блока кода первого истинного выражения, последующие условия case не вычисляются, а соответствующие инструкции не выполняются.

Блок case _ не обязателен и выполнится, если ни одно условие в других case не будет верным. Он аналогичен блоку default оператора switch case С-подобных языков программирования.

Пример:

```

1 | match name:
2 |     case 'red':
3 |         color = '#f00'
4 |     case 'green':
5 |         color = '#0f0'
6 |     case _:
7 |         color = '#00f'

```

Проверка состояния — это не просто точное совпадение переменной с указанным в case значением, а сопоставление с шаблоном (pattern matching statement). Это целый механизм для проверки данных, их распаковки и управления потоком выполнения.

Пример:

```

1 | match direction:
2 |     case 'North' | 'East' | 'South' | 'West':
3 |         print('Хорошо, я пошел!')
4 |     case _:
5 |         print('Неизвестное направление...')

```

Здесь «|» означает «или».

Сравнивать при этом можно и более сложные структуры:

```

1 | command = ('Inc', 25)
2 | value = 5
3 |

```

```

4 | match command:
5 |     case 'Inc', n:
6 |         value+=n
7 |     case 'Dec', n:
8 |         value-=n
9 |
10 | print(result) #30

```

В данном примере происходит сравнение нулевого элемента (строки) кортежа `command`, первый же элемент (число) ассоциируется (*захватывается*) с `n` и участвует в дальнейших операциях.

Также данный оператор позволяет проверить *соответствие* некоторому *шаблону*:

```

1 | match c:
2 |     # если c представляет из себя тройку значений
3 |     case (r, g, b):
4 |         color='#%02x'%02x'%02x'%(r,g,b)
5 |     # если c — атомарное значение
6 |     case str_code:
7 |         color=str_code.lower()

```

В данном примере в строке 4 указана строка формата, представляющая входящие десятичные числа `r`, `g`, `b` в шестнадцатеричном виде, каждое длины 2.

Еще одним полезным примером может оказаться проверка на `None`:

```

1 | match name:
2 |     case None:
3 |         print('Hello there')
4 |         color='#%02x'%02x'%02x'%(r,g,b)
5 |     case some_name:
6 |         print('Hello '+some_name)

```

12. Перечисления Enum

Перечисления — это наборы символических имен, связанных с уникальными константными значениями. Они используются

для создания простых пользовательских типов данных, таких как времена года, недели, виды оружия в игре, планеты, оценки или дни. По соглашению имена перечислений начинаются с заглавной буквы и употребляются в единственном числе.

Для создания перечислений в Python используется модуль `enum`. Пример:

```
1 | from enum import Enum
2 |
3 | class Turff(Enum):
4 |     SOIL = 1
5 |     SAND = 2
6 |     WATER = 3
7 |     ROCKY = 4
8 |     NONE = 0
```

Для создания перечисления использовано слово `class`. После этого можно использовать данное перечисление:

```
9 | t = Turff.SAND
10 | print(t)
```

В результате на консоль будет напечатано «Turff.SAND».

Получить список всех значений перечисления можно функцией приведения к списку:

```
11 | print(list(Turff))
```

Проверить, содержит ли переменная значение из заданного перечисления, можно функцией `isinstance`:

```
12 | print(isinstance(t, Turff)) # True
```

Получить доступ к полю из перечисления можно как по его названию, так и по его значению:

```
13 | print(Turff(0)) # Turff.NONE
14 | print(Turff['WATER']) # Turff.WATER
```

Вместо ручного указания численных значений в строках 4-8 можно использовать функцию `auto` (ее также надо импортировать из `enum`).

Стоит отметить, что значения, присвоенные в строках 4-8 в общем случае не обязаны быть уникальными, однако члены перечисления имеющие одинаковые значения не различимы между собой. Так, если значение `SAND` поменять на `1` и попытаться выполнить строки 9-10 или вывести значения перечисления для единицы, подобно строке 13, то будет напечатано «Turff.SOIL».

Для создания перечисления необязательно использовать класс. Альтернативный способ создания представляет из себя вызов функции:

```
1 | from enum import Enum
2 |
3 | Chance = Enum('Chance',
4 |               'LOW MEDIUM HIGH', start=1)
```

При этом переменная `Chance` будет выступать как имя перечисления в коде:

```
5 | t = Chance.LOW
```

В то время как строка `'Chance'` будет указана при печати:

```
6 | print(t)
```

Вывод: «`Chance.LOW`» — именно это слово (до точки) и было выведено. То есть можно указать другое название для вывода, например, «Вероятность».

Сами перечисляемые значения задаются в строке, разделенные пробелами. Число, переданное в `start`, определяет начало нумерации значений для членов перечисления.

13. Задачи

13.1. Примеры задач

Пример 1. Написать функцию, разделяющую некоторую последовательность на две части: до заданного *n* и после. Для демонстрации работы, необходимо реализовать консольное приложение с меню.

Решение:

```
1 | def split_by(sequence, n):
2 |     return sequence[:n], sequence[n:]
3 |
4 | s = ""
5 | while True:
6 |     print("Текущая строка '" + s + "'", sep="")
7 |     i = int(input("[1] - Изменить строку
8 |                  [2] - Ввести n и выполнить
9 |                  [0] - Выход\n"))
10 |    match i:
11 |        case 0: break;
12 |        case 1: s = input('Введите строку: ')
13 |        case 2:
14 |            n = int(input('Введите число: '))
15 |            res = split_by(s, n)
16 |            print('Разделенные строки:',
17 |                res[0], res[1], sep="\n")
```

Сама функция занимает первые две строки кода. Решение выполняется путем операции нарезания. Оператор `return` возвращает сразу два значения (один кортеж значений).

Далее реализовано меню с помощью «бесконечного» цикла `while` и оператора множественного выбора `match-case` (для версий Python 3.10 и выше). Выход из цикла происходит благодаря оператору `break`.

Вызов требуемой функции производится в строке 16.

Пример 2. Написать класс «газировка», с полем «ингредиент». Класс имеет метод `get_info`, который выводит с каким газировка

дополнительным ингредиентом, либо, если его нет, выводит информацию о том, что это обычная газировка.

Написать класс «автомат газированной воды», который имеет набор газировок и цен, соответствующих им. Он может предоставить всю информацию о продаваемых газировках, добавить новый вид газировки, а также проверить, хватит ли введенной суммы, для покупки выбранной газировки.

Создать консольное приложение с меню, демонстрирующее работу автомата с газировкой.

```
1 class Soda:
2     __ingr = None
3
4     # Конструктор
5     def __init__(self, ingredient: str = None):
6         self.__ingr = ingredient
7
8     def get_info(self):
9         if self.__ingr is None:
10             return 'Обычная газировка'
11         else:
12             return 'Газировка с добавкой "' + self.__ingr + '"'
13
14 class SodaMachine:
15     __prices = dict() # Названия и цены
16
17     def add_soda(self, soda, price):
18         self.__prices[soda] = price
19
20     def get_prices(self):
21         return self.__prices
22
23     def chek_price(self, soda, money):
24         if not (soda in self.__prices):
25             return False
26         real_price = self.__prices[soda]
27         return money >= real_price
```

```

28
29 def add(sm):
30     ingredient = input('Введите добавку: ')
31     if ingredient == "":
32         s = Soda()
33     else:
34         s = Soda(ingredient)
35     p = int(input('Введите цену: '))
36     sm.add_soda(s, p)
37
38 def show(sm):
39     for t in sm.get_prices().items():
40         print('{0:40} {1:>5}'.format(t[0].get_info(), t[1]))
41
42 def chek(sm):
43     # выбор газировки по названию
44     sodas = sm.get_prices()
45     show(sm)
46     name = input('Введите название газировки: ')
47     my_soda = None
48     for s in sodas.keys():
49         if s.get_info() == name:
50             my_soda = s
51     if my_soda is None:
52         print ('Нет такой газировки')
53         return
54     # проверка цены
55     p = int(input('Введите цену: '))
56     res = sm.chek_price(s, p)
57     if res:
58         print('Денег достаточно')
59     else:
60         print('Денег мало')
61
62 # main
63 soda_machine = SodaMachine()

```

```

64 | while True:
65 |     print('[1] - Добавить газировку')
66 |     print('[2] - Проверить возможность покупки')
67 |     print('[3] - Напечатать все')
68 |     print('[0] - Выход')
69 |     c = int(input())
70 |     if c == 1:
71 |         add(soda_machine)
72 |     elif c == 2:
73 |         chek(soda_machine)
74 |     elif c == 3:
75 |         show(soda_machine)
76 |     elif c == 0:
77 |         break

```

Сначала описан класс газировки Soda, который имеет одно закрытое поле `__ingr`, открытый конструктор и открытый метод `get_info`.

Конструктор принимает один аргумент `ingredient` со значением по умолчанию `None` (то есть, если значение аргумента не будет указано, оно будет равно `None`).

Метод `get_info` возвращает строку, которая описывает данную газировку, в зависимости от добавочного ингредиента и от наличия этого ингредиента.

Далее описан класс автомата с газировкой `SodaMachine`, который хранит словарь соответствий вида «газировка-цена». Он имеет три открытых метода.

Первый, `add_soda`, добавляет соответствующую газировку и ее цену в словарь. Если газировка уже была, значение цены обновится. Важно при этом, что сравнивается объект класса `Soda`, а не строковое название газировки. То есть, теоретически, могут быть две разные газировки с одинаковым описанием.

Метод `get_prices` просто предоставляет доступ к хранимому словарю.

Метод `chek_price` проверяет, достаточно ли указанной суммы для покупки указанной газировки. При этом опять сравниваются объект класса `Soda`.

Далее удобнее рассмотреть сначала основную часть кода (начиная со строки 62). Здесь реализуется меню: осуществляется взаимодействие консоли и одного экземпляра автомата с газировкой. Данный код разбит на три метода: `add`, `chek` и `show`, каждый из которых вызывает соответствующие методы класса `SodaMachine`.

Метод `add` (строка 29) осуществляет ввод газировки и ее цены и последующее добавление этой информации в автомат. Если в качестве добавки была введена пустая строка, будет добавлена газировка без добавок.

Метод `show` (строка 38) осуществляет форматированный вывод словаря.

Метод `chek` (строка 42) сначала предоставляет выбор газировки, а затем проверяет, продадут ли ее за указанную цену. Для выбора газировки осуществляется сопоставление пользовательского ввода с описанием всех газировок. Если совпадения нет, то выведется информация «Нет такой газировки» (при этом `return` без значений завершит выполнение функции, то есть дальнейший код этой функции не будет выполнен, если газировка не найдена), иначе будет осуществлена проверка, посредством вызова ранее описанного метода `chek_price` класса `SodaMachine`.

Пример 3. Написать консольную программу с меню, преобразующую вводимую строку в целое число с помощью метода приведения типов `int`. Строка может содержать любые символы, а также может быть пустой. Задача не должна выпадать с ошибкой ни при каких обстоятельствах, но должна выводить информацию об ошибке на консоль.

Решение:

```
1 | while True:
2 |     print('Для завершения введите "Q"')
3 |     s = input('Введите строку: ')
4 |     if s == 'Q':
5 |         break
6 |     try:
7 |         i = int(s)
8 |         print('Введено число:', i, '\n')
```

```

9 | except ValueError as e:
10 |     print('Ошибка:', e, '\n')

```

Опасный участок кода в данном случае — приведение строки к типу `int`. Поэтому в блок `try` помещается все, начиная от этой строки и все последующие строки, которые не должны быть выполнены в случае, если преобразование вызовет исключение (то есть строка 8 с выводом числа).

Оператор `except` отлавливает исключения только необходимого типа `ValueError`.

Пример 4. Написать функцию возведения целого числа в целую степень. Написать функцию, которая склеивает две строки. Написать декоратор, который меняет местами аргументы функции. Декорировать обе функции. Разработать консольное приложение с меню, демонстрирующее их работу.

Решение:

```

1 | def MadDecorator(func):
2 |     def wrapper(fst, snd):
3 |         return func(snd, fst) # меняются
4 |     return wrapper
5 |
6 | @MadDecorator
7 | def Pow(x, n):
8 |     return x ** n
9 |
10 | @MadDecorator
11 | def Conc(s1, s2):
12 |     return s1 + s2
13 |
14 | while(True):
15 |     print('Введите пустую строку для завершения')
16 |     print('или любую другую, чтобы продолжить')
17 |     cont = input()
18 |     if not cont:
19 |         break
20 |     # Ввод

```



```

21 | print('Введите число 1')
22 | x = int(input())
23 | print('Введите число 2')
24 | y = int(input())
25 | print('Введите строку 1')
26 | s1 = input()
27 | print('Введите строку 2')
28 | s2 = input()
29 | # Вывод
30 | p = Pow(x, y)
31 | print('%d ^ %d = %d' % (x, y, p))
32 | c = Conc(s1, s2)
33 | print("'" + s1 + " " + s2 + "'" + " = '" + c + "'")

```

Сначала (строки 1-4), описан декоратор MadDecorator, который применим к функциям с двумя аргументами и меняет эти аргументы местами.

Далее описаны две простейшие функции: Pow (строки 6-8) и Conc (строки 10-12) с этим декоратором.

После чего реализовано меню. При этом, данные в меню выводятся так, как если бы функции Pow и Conc были написаны правильно (без декоратора). Например: « $2 \wedge 10 = 100$ » и «"ка" + "чай" = "чайка"».

13.2. Задачи на функции

Написать требуемые функции и консольное приложение с меню, демонстрирующее их работу.

1. Ввод целого числа из диапазона $[a;b]$, где a и b — параметры функции. Введенное число — ее результат.

2. Вывод элементов последовательности в n столбцов, где n и последовательность — параметры функции.

3. Функция, принимающая имя файла как параметр, и читающая этот файл в список построчно. Если файла нет, возвращает None, если есть — список.

4. Функция принимает число n и кортеж чисел. В результате работы функция возвращает два значения: список, все элементы которого $\geq n$; количество не попавших в этот список значений.

5. Первая функция принимает на вход последовательность и некую функцию. Она вызывает входную функцию для каждого элемента последовательности. Вторая функция имеет один аргумент и просто печатает его 5 раз. Эта функция и должна передаваться в первую.

6. Функции `sum_range` и `mult_range`, которые принимают три целочисленных параметра — начало, конец и шаг, а возвращают соответственно сумму и произведение всех целых чисел от начала до конца с заданным шагом.

7. Функция получает на вход строку и возвращает три значения: количество букв в верхнем регистре, количество букв в нижнем регистре, количество прочих символов.

8. Функция получает на вход строку и возвращает новую строку, состоящую из уникальных символов входящей (то есть каждый элемент по одному разу). При этом регистр учитываться не должен, то есть «А» = «а».

9. Функция принимает на вход целое число `n` и кортеж. В результате работы функция возвращает два значения: кортеж из кратных `n` элементов; количество не попавших в этот кортеж значений.

10. Функция проверяет, является ли входящая строка палиндромом или нет. При этом регистр учитываться не должен, то есть «А» = «а».

11. Функция 1 принимает на вход кортеж чисел и функцию 2. Она вызывает функцию 2 для каждого элемента кортежа. Функция 2 возводит число в куб и печатает результат.

12. Функция принимает на вход кортеж чисел. Она возвращает кортеж, состоящий только из тех чисел входного, которые кратны некоторой целочисленной глобальной константе `INTVAL`, при этом эти числа идут в обратном порядке, относительно входного кортежа.

13. Функция принимает на вход строку. Каждый раз, когда она встречает символ в верхнем регистре, она увеличивает некую глобальную переменную `num_upper`.

13.3. Задачи на ООП

Создать консольное приложение с меню, демонстрирующее работу следующих задач:

1. Написать класс «автомобиль», с полями «марка», «мощность двигателя», «тип кузова». Написать класс «база автомобилей», который хранит набор автомобилей и имеет методы, возвращающие: кортеж автомобилей по указанной марке, кортеж автомобилей по указанному типу кузова, кортеж автомобилей, мощность двигателя которых попадает в указанный интервал, а также предоставляет информацию о всех автомобилях для дальнейшей печати и позволяет пополнять базу новыми авто.

2. Написать класс «сотрудник», с полями «ФИО» и «зарплата». Написать класс «отдел», который хранит набор сотрудников и имеет методы, позволяющие: принять сотрудника, уволить сотрудника, повысить сотруднику зарплату, получить информацию о всех сотрудниках для дальнейшей печати, посчитать среднюю зарплату в отделе, посчитать суммарную зарплату в отделе, получить только ФИО всех сотрудников.

3. Написать класс «студент», который хранит ФИО и информацию о пяти оценках за сессию. Данный класс переопределяет операцию сравнения — больше тот, у кого средний бал выше. Написать класс «успеваемость группы», который хранит набор студентов и имеет методы, позволяющие: добавить студента вместе с его успеваемостью, получить информацию о всех студентах для дальнейшей печати, вывести количество студентов в группе, имеющих задолженности (отметка меньше тройки), вывести студента с лучшей по группе успеваемостью.

4. Написать класс «воин», с полями «имя», «здоровье», «урон» и «разброс урона» и методом «атаковать», который наносит указанному воину случайный урон из промежутка [урон - разброс урона; урон + разброс урона]. Написать класс «арена» с методами «бой», который возвращает победителя среди двух, переданных в этот метод, воинов (кто бьет первым выбирается случайным образом, здоровье воинов после боя восстанавливается до исходного значения) и метод «турнир», который получает на вход последовательность из 8 воинов и возвращает кортеж из 8 занятых

ими мест в турнире. Пользователь должен иметь возможность создать воинов, провести турнир и посмотреть турнирную сетку.

5. Написать класс «приз», с полями «название» и «стоимость». Написать класс «игровой автомат», который хранит набор призов и имеет возможность добавить приз, получить информацию о всех призах для дальнейшей печати, а также метод, позволяющий дернуть рычаг однорукого бандита. Метод возвращает приз, либо None, в случае неудачи. Обозначим $S = N * \text{сумма стоимости всех призов}$. Шанс выпадения определенного приза равен $S / (\text{стоимость этого приза})$. То есть первый приз будет получен, если случайная величина от 0 до S попадет в промежуток [0; стоимость 1), второй — в промежуток [стоимость 1; стоимость 2), третий — в промежуток [стоимость 2; стоимость 3) и так далее. Суммарный шанс на победу таким образом должен составить $1/N$.

6. Создать класс «туча», который имеет два поля — тип осадков и их количество. Тип осадков при этом задается строкой, например, «*» для снега, «'» для дождя, «"» для ливня... Класс перегружает метод приведения к строке, выводя заданный тип осадков заданное число раз (Например, «*****» для пяти снежинок). Создать класс «небо», который хранит набор туч. В этом наборе, вместо туч, могут стоять значения None, означающие ясный участок неба (шириной 5 символов). Класс имеет методы добавления туч, добавления ясного участка, удаления тучи или ясного неба по индексу и метод отображения неба в консоли (печать всех туч и ясных областей).

7. Создать класс «собака», который имеет два поля — имя и громкость лая в децибелах. Создать класс «зверинец», содержащий набор собак. Класс имеет методы: получения информации о всех собаках для дальнейшей печати, добавления собаки, метод, позволяющий лаять собаке, по введенному пользователем имени, и метод, вычисляющий шум в децибелах, от одновременного лая всех собак (интенсивность шума в децибелах вычисляется вовсе не простым сложением, например, $10\text{дБ} + 10\text{дБ} = 13\text{дБ}$).

8. Создать класс «боец» с полями «имя», «текущее здоровье», «максимальное здоровье», «урон» и «скорость бега». Создать класс «полководец» с отрядом бойцов и полем, хранящим его доблестное

имя. Полководец должен мочь посчитать скорость движения отряда (по самому медленному бойцу), число воинов в отряде, число раненых воинов отряда (здоровье ниже заданного процента), а также некоторую величину боеспособности (суммарный урон * сумма текущего здоровья), и максимальной боеспособности (суммарный урон * сумма максимального здоровья) для оценки своих сил по сравнению с другими отрядами. Кроме того, должна быть реализована возможность рекрутировать воина и вывести подробную информацию о каждом бойце отряда на экран.

9. Создать класс «база», хранящий названия товаров и их количество в тоннах, а также журнал поставок. Создать класс «фура» с полями «название», «товар», «грузоподъемность». На базу может приехать груженная фура (всегда полная), и тогда количество указанного товара на базе увеличивается (или добавляется новый), или пустая фура, готовая увести заданный товар. Товар может полностью закончиться и тогда должен быть удален. При каждом завозе или вывозе подробная информация записывается в журнал. Пользователь должен иметь возможность привести товар, вывести, посмотреть информацию о хранимом на базе товаре, посмотреть журнал.

10. Написать класс «дом» с полями «адрес» и «число жильцов», методом, позволяющим изменить число жильцов и перегрузкой метода приведения к строке. Написать класс «город», хранящий набор домов. Должны быть реализованы возможности построить дом, вывести информацию о всех домах, посчитать число жильцов в городе, а также метод «прожить один год», за который в каждом доме рождается и умирает случайное число человек (от нуля до половины жителей).

13.4. Задачи на исключения

1. В коде дан список неоднородных элементов. Написать консольную программу, вычисляющую корни всех элементов списка функцией `math.sqrt` и печатающую их. Задача не должна выпадать с ошибкой, но должна выводить информацию о ней на консоль.

2. Написать программу ввода целого числа. Если ввод пользователя не удастся преобразовать к числу, выводится

информация об ошибке и запрос на ввод повторяется до тех пор, пока не будет введена подходящая строка.

13.5. Задачи на декораторы

1. Написать функцию, проверяющую, начинается ли строка с заглавной буквы. Написать функцию, которая удаляет из строки все цифры. Создать декоратор, который приводит строку к верхнему регистру перед вызовом функции. Декорировать им обе функции. Разработать консольное приложение с меню, демонстрирующее их работу.

2. Написать функцию возведения в квадрат. Написать функцию, отсекающую у строки первый и последний символ. Создать декоратор, который не просто вызывает метод от указанного параметра, но и вызывает ее повторно от полученного результата. Декорировать им обе функции.

3. Написать функцию, которая преобразует строку в список символов. Написать функцию, которая удаляет каждый второй символ строки. Создать декоратор, который разворачивает результат функции (последовательность возвращается в обратном порядке). Декорировать им обе функции.

4. Написать функцию, которая вычисляет среднее арифметическое списка. Создать декоратор, который увеличивает результат функции на 2. Создать декоратор, который увеличивает результат функции в 2 раза. Декорировать обоими функциями.

5. Написать функцию, которая принимает текст и исправляет первый символ в каждом предложении на заглавный (текст утвердительный). Создать декоратор, который заменяет слово «дурак» на звездочки. Создать декоратор, который начинает каждое предложение с новой строки. Декорировать ими обоими функциями.

6. Написать функцию, которая вычисляет площадь треугольника по трем сторонам. Создать декоратор, который округляет результат до одного знака после запятой. Создать декоратор, который печатает все аргументы функции на консоль, при ее вызове. Декорировать ими обоими функциями.

Список литературы

1. Лутц М. Изучаем Python Том 1. 5-е издание = Learning Python, 5th Edition. – ООО “Диалектика”, 2019. -833 с.
2. Python. Статические свойства (поля, элементы класса) кода – URL: <https://dev-gang.ru/article/kommentirovanie-python-koda-auf6lgv2vv/> (дата обращения: 16.10.2023).
3. Приватные методы без нижнего подчеркивания и интерфейсы в Python – URL: <https://habr.com/ru/articles/443192/> (дата обращения: 26.10.2023).
4. Статические переменные и методы в Python – URL: <https://pythonpip.ru/osnovy/staticheskie-peremennye-i-metody-v-python> (дата обращения: 16.10.2023).
5. Перегрузка операторов – URL: <https://pythonworld.ru/osnovy/peregruzka-operatorov.html> (дата обращения: 15.10.2023).
6. Объектно-ориентированное программирование в Python – URL: <https://codechick.io/tutorials/python/python-what-is-oop> (дата обращения: 15.10.2023).
7. Базовые математические функции – URL: <https://proproprogs.ru/modules/numpy-bazovye-matematicheskie-funkcii> (дата обращения: 01.10.2023).
8. Изменить размер рисунка в Matplotlib – URL: <https://dev-gang.ru/article/izmenit-razmer-risunka-v-matplotlib-9btqsjj4ff/> (дата обращения: 01.10.2023).
9. Как использовать функцию NumPy `arange()` в Python – URL: <https://andreyex.ru/programmirovaniye/python/kak-ispolzovat-funktsiyu-numpy-arange-v-python/> (дата обращения: 01.10.2023).
10. NumPy, часть 1: начало работы – URL: <https://pythonworld.ru/numpy/1.html> (дата обращения: 01.10.2023).
11. Библиотека Numpy. Полезные инструменты – URL: <https://devpractice.ru/numpy-useful-functions/> (дата обращения: 01.10.2023).
12. Руководство по использованию Python-библиотеки NUMPY – URL: <https://pythonru.com/biblioteki/rukovodstvo-po-ispolzovaniju-python-biblioteki-numpy> (дата обращения: 01.10.2023).

13. Гипербола и биквадратные уравнения – URL: https://www.mathnet.ru/php/archive.phtml?wshow=paper&jrnid=mo&paperid=627&option_lang=rus (дата обращения: 30.09.2023).
14. Уравнения кривых. Лемниската Бернулли – URL: <https://www.calc.ru/Uraveniya-Krivykh-Lemniskata-Bernulli.html> (дата обращения: 30.09.2023).
15. Кардиоиды – URL: <http://school-collection.edu.ru/catalog/res/5c484b99-e427-4a7b-89a6-60e64f6c7426/view/> (дата обращения: 30.09.2023).
16. Биквадратная кривая – URL: <https://mash-xxl.info/info/28249/> (дата обращения: 30.09.2023).
17. Полярные цветы – URL: <http://blog.kislenko.net/show.php?id=1891> (дата обращения: 27.09.2023).
18. Декораторы – URL: https://ru.hexlet.io/courses/python-functions/lessons/decorators/theory_unitc (дата обращения: 29.10.2023).
19. Функция hex() в Python, число в шестнадцатеричную строку – URL: <https://docs-python.ru/tutorial/vstroennye-funktsii-interpretatora-python/funktsiya-hex/> (дата обращения: 29.10.2023).
20. Соответствие структуре шаблона, конструкция match/case – URL: <https://docs-python.ru/tutorial/tsikly-upravlenie-vetvleniem-python/konstruktsiya-match-case/> (дата обращения: 29.10.2023).
21. Конструкция match-case в Python – URL: <https://pythonist.ru/konstrukciya-match-case-v-python-polnoe-rukovodstvo/> (дата обращения: 29.10.2023).
22. Перечисления enum в Python – URL: <https://pythonru.com/osnovy/perechisleniya-enum-v-python> (дата обращения: 29.10.2023).